

Кременчуцький національний університет імені Михайла  
Остроградського  
Навчально-науковий інститут електричної інженерії та інформаційних  
технологій  
Кафедра комп'ютерної інженерії та електроніки

## **ПОЯСНЮВАЛЬНА ЗАПИСКА**

до кваліфікаційної роботи

бакалавр

на тему: «AI-система семантичного пошуку в корпусі освітніх  
документів на основі LLM з інтерактивною графовою навігацією  
результатів»

Виконав: студент 4 курсу групи КІ-22-1

ступінь вищої освіти бакалавр

спеціальність 123 «Комп'ютерна інженерія»

освітня програма «Комп'ютерна інженерія»

Ілля ГРИЩЕНКО

Керівник \_\_\_\_\_

Рецензент \_\_\_\_\_

м. Кременчук 2026 року

## РЕФЕРАТ

**Грищенко І. В. AI-система семантичного пошуку в корпусі освітніх документів на основі LLM з інтерактивною графовою навігацією результатів.**

Спеціальність 123 – Комп'ютерна інженерія, освітньо-професійна програма – Комп'ютерна інженерія – КрНУ. – Кременчук, 2026.

Об'єктом розробки є програмна система пошуку та навігації в корпусі освітніх документів.

Метою роботи є розроблення веб-системи EduGraph, яка синхронізує документи з Google Drive, виконує семантичний пошук за змістом і відображає результати у вигляді інтерактивного графу.

У роботі проаналізовано альтернативні технологічні рішення, спроектовано рольові сценарії та модель даних, реалізовано ASP.NET Core API, FastAPI-сервіс із multilingual-e5, LanceDB і CrossEncoder, а також React-клієнт із графовою візуалізацією. Описано синхронізацію Google Drive, парсинг документів, індексацію, пошук, перевірку сценаріїв та порядок роботи користувача.

Робота містить \_\_\_\_ сторінок, 3 основні розділи, 14 рисунків, 43 таблиць, 20 джерел і 5 додатків.

**Ключові слова:** семантичний пошук, embeddings, reranking, ASP.NET Core, React, FastAPI, Google Drive, LanceDB, графова навігація.

## ABSTRACT

The object of development is a software system for searching and navigating a corpus of educational documents. The purpose is to develop EduGraph, a web system that synchronizes documents from Google Drive, performs semantic retrieval, and presents results as an interactive graph. The work covers technology selection, data design, ASP.NET Core API, a FastAPI retrieval service with multilingual-e5, LanceDB and CrossEncoder reranking, and a React client with graph visualization.

**Keywords:** semantic search, embeddings, reranking, ASP.NET Core, React, FastAPI, Google Drive, LanceDB, graph navigation.

**ЗАВДАННЯ НА КВАЛІФІКАЦІЙНУ РОБОТУ**

Студент: Грищенко Ілля Володимирович.

Тема кваліфікаційної роботи: «AI-система семантичного пошуку в корпусі освітніх документів на основі LLM з інтерактивною графовою навігацією результатів».

Керівник кваліфікаційної роботи: \_\_\_\_\_.

Затверджено наказом від \_\_\_\_\_ 2026 року № \_\_\_\_\_.

Строк подання студентом кваліфікаційної роботи: \_\_\_\_\_.

Вихідні дані: монорепозіторій EduGraph, корпус освітніх документів у Google Drive, вимоги до функціональності й оформлення роботи.

Зміст роботи: аналіз предметної області; функціональна модель; проєктування БД; реалізація семантичного пошуку; графове подання; ASP.NET Core API; синхронізація Google Drive; інтерфейс; тестування; інструкція користувача.

Перелік графічного матеріалу: архітектура системи; діаграма варіантів використання; логічна й фізична схеми БД; алгоритми семантичного пошуку та синхронізації; модель графowego подання; екрани інтерфейсу.

Дата видачі завдання: \_\_\_\_\_.

Студент \_\_\_\_\_ Ілля ГРИЩЕНКО

Керівник роботи \_\_\_\_\_

## КАЛЕНДАРНИЙ ПЛАН

| № | Назва етапу                                            | Строк виконання | Примітка |
|---|--------------------------------------------------------|-----------------|----------|
| 1 | Отримання завдання                                     | _____           |          |
| 2 | Аналіз предметної області та альтернатив               | _____           |          |
| 3 | Проектування архітектури, БД і сценаріїв               | _____           |          |
| 4 | Реалізація ASP.NET Core API та Google Drive інтеграції | _____           |          |
| 5 | Реалізація FastAPI-сервісу семантичного пошуку         | _____           |          |
| 6 | Реалізація React-клієнта та графового подання          | _____           |          |
| 7 | Функціональна перевірка системи                        | _____           |          |
| 8 | Оформлення записки й графічних матеріалів              | _____           |          |
| 9 | Подання роботи на перевірку та захист                  | _____           |          |

Студент \_\_\_\_\_ Ілля ГРИЩЕНКО

Керівник роботи \_\_\_\_\_

**ВІДОМІСТЬ РОБОТИ**

| Формат | Позначення                        | Найменування            | Аркушів | Примітка |
|--------|-----------------------------------|-------------------------|---------|----------|
| A4     | КрНУ.26.ІЕЛПТ.<br>123.____.001.ПЗ | Записка<br>пояснювальна | —       |          |

## **ЗМІСТ**

Зміст оновлюється автоматично під час відкриття документа.

## ВСТУП

Цифрове освітнє середовище університету містить освітні програми, методичні рекомендації, робочі програми дисциплін, інструкції, звіти та допоміжні матеріали. Наявність файла у сховищі не гарантує, що користувач швидко знайде потрібний фрагмент.

Пошук за точним збігом слів обмежений різними формулюваннями одного поняття та великим обсягом документів. Семантичний пошук порівнює векторні подання запиту й текстових фрагментів, що дозволяє враховувати змістову близькість [2, 3].

Тема роботи – «AI-система семантичного пошуку в корпусі освітніх документів на основі LLM з інтерактивною графовою навігацією результатів». Реалізація EduGraph не генерує відповідь чат-моделлю. Transformer-моделі використовуються для побудови embeddings і повторного ранжування знайдених фрагментів; користувач отримує уривок та посилання на оригінальний документ [1, 4].

Метою роботи є розроблення веб-системи EduGraph, яка забезпечує синхронізацію корпусу з Google Drive, збереження метаданих у SQLite, побудову векторного індексу, семантичний пошук і графове подання результатів.

Об'єктом розробки є процес пошуку та навігації в корпусі освітніх документів. Предметом розробки є програмні засоби синхронізації, витягування тексту, індексації, ранжування та клієнтської візуалізації.

Для досягнення мети необхідно проаналізувати альтернативні рішення; сформулювати вимоги; спроектувати ролі, БД і взаємодію компонентів; описати реалізацію ASP.NET Core API, FastAPI-сервісу й React-клієнта; перевірити основні сценарії та підготувати інструкцію користувача.

Практична цінність EduGraph полягає у використанні наявного Google Drive як джерела документів без перенесення офіційних матеріалів до нового сховища. Система додає до чинної структури семантичний пошук і графову навігацію.

# **1 АНАЛІТИЧНА ЧАСТИНА**

## **1.1 Аналіз існуючих типів застосунків для пошуку в освітніх документах**

Освітні документи можуть зберігатися в різних інформаційних системах, і кожен клас систем має власну логіку пошуку. Найпростішим класом є файлові сховища, до яких належать Google Drive, OneDrive або локальні мережеві каталоги. Вони добре виконують функцію збереження файлів, надання спільного доступу, організації папок і контролю посилань. Для університетського середовища це важливо, оскільки матеріали часто оновлюються різними відповідальними особами й повинні залишатися доступними за стабільними посиланнями.

Недоліком файлових сховищ є те, що їхній пошук орієнтований переважно на назви, метадані або простий повнотекстовий індекс. Якщо документ має загальну назву або містить велику кількість розділів, користувачеві складно зрозуміти, де саме знаходиться потрібний фрагмент. Також файлове сховище не завжди пояснює релевантність результату: користувач бачить файл, але не бачить, який саме уривок відповідає запиту.

Другим класом є системи управління навчанням. Вони добре структурують матеріали за курсами, темами, завданнями й календарем освітнього процесу. Однак LMS зазвичай прив'язана до конкретної дисципліни або групи, а не до наскрізного корпусу документів. Якщо потрібно знайти матеріал, що стосується кількох дисциплін, навчальних планів або нормативних документів, користувач може не знати, в якому курсі або розділі його шукати.

Третій клас становлять системи електронного документообігу. Вони орієнтовані на маршрутизацію документів, погодження, контроль версій і формальні процеси. Такі системи важливі для адміністративної діяльності, але не завжди зручні для студентського або викладацького пошуку навчального змісту. Їхній інтерфейс часто побудований навколо реквізитів документа, а не навколо змістового запиту користувача.



Четвертим класом є повнотекстові пошукові системи. Вони будують індекс термінів і дозволяють швидко знаходити документи, де зустрічаються певні слова. Такі системи можуть бути дуже ефективними для точних запитів, наприклад для пошуку назви нормативного документа або конкретного терміна. Однак вони слабше працюють там, де користувач формулює запит іншими словами, ніж автор документа.

Для освітніх матеріалів ця проблема є типовою. Один і той самий зміст може бути описаний через різні поняття: «результати навчання», «компетентності», «очікувані результати», «програмні результати». Якщо система не враховує семантичну близькість, вона може не знайти релевантний фрагмент або поставити його нижче менш корисного документа. Тому для задачі EduGraph важливо шукати не тільки збіг символів, а й змістову відповідність.

П'ятим класом є системи семантичного пошуку. Вони представляють текст у вигляді числових векторів, які відображають зміст. Запит користувача також перетворюється на вектор, після чого система шукає найближчі фрагменти у векторному просторі. На відміну від класичного пошуку, такий підхід може знайти документ навіть тоді, коли слова запиту й документа не збігаються повністю.

Семантичний пошук особливо корисний тоді, коли корпус складається з довгих документів. Якщо індексувати документ цілком, релевантність може розмиватися через велику кількість тем усередині одного файлу. Тому EduGraph використовує підхід з поділом документа на фрагменти. Кожен chunk індексується окремо, а користувач отримує не тільки назву документа, а й текстовий уривок, який став причиною релевантності.

Окремою вимогою є навігація по результатах. Звичайний список документів показує порядок релевантності, але не показує структуру корпусу. Якщо результати належать до різних папок, користувачу корисно бачити, чи зосереджені вони в одному тематичному розділі або розкидані по різних частинах сховища. Тому EduGraph доповнює пошук графовим поданням, де

папки та документи є вузлами, а зв'язки відображають належність документа до певної частини корпусу.

Таким чином, аналіз існуючих типів застосунків показує, що жоден окремих клас традиційних систем не закриває повністю потребу EduGraph. Файлове сховище потрібне як джерело документів, серверна система потрібна для контролю доступу, semantic search потрібний для змістового пошуку, а графове подання потрібне для навігації. Саме поєднання цих підходів визначає архітектуру розроблюваної системи.

## **1.2 Аналіз типової архітектури веб-системи семантичного пошуку**

Типова веб-система семантичного пошуку складається з кількох логічних шарів. Перший шар – клієнтський інтерфейс, через який користувач вводить запит, переглядає результати та виконує дії відповідно до своєї ролі. Другий шар – backend API, який приймає запити, перевіряє авторизацію, виконує бізнес-логіку та координує роботу інфраструктурних компонентів. Третій шар – сховище структурованих даних, де зберігаються користувачі, документи, стани індексації та службова інформація.

Для системи семантичного пошуку потрібен також окремих індексаційний шар. Він відповідає за перетворення документів у придатний для пошуку вигляд: отримання тексту, нормалізацію, поділ на фрагменти, побудову embeddings і збереження векторів. Цей шар відрізняється від звичайної бізнес-логіки, оскільки використовує ML/NLP-бібліотеки, великі моделі та спеціалізоване векторне сховище.

У EduGraph клієнтським шаром є React SPA. Він не містить логіки синхронізації документів або виконання embeddings, а працює з backend через HTTP. Такий підхід зменшує складність клієнта й не відкриває внутрішню інфраструктуру напряду. Клієнт отримує тільки ті дані, які потрібні для інтерфейсу: список папок, результати пошуку, фрагменти, посилання та інформацію про роль користувача.

Backend-шар реалізований на ASP.NET Core Web API. Він виконує роль центрального координатора. Саме backend знає про користувачів, ролі, заявки,

Google Drive, SQLite і FastAPI-сервіс. Клієнт ніколи не звертається напряму до Python-сервісу, тому всі пошукові операції проходять через контроль авторизації. Це важливо для університетського середовища, де доступ до матеріалів має бути керованим.

Сховище структурованих даних реалізовано через SQLite. У ньому зберігаються користувачі, ролі, заявки на реєстрацію, папки Google Drive, документи, hash вмісту та статуси індексації. SQLite не використовується для векторного пошуку, але є джерелом істини для доменних метаданих. Це розділення дозволяє не змішувати реляційні дані з ML-індексом.

Векторний пошук реалізований окремим FastAPI-сервісом. Він використовує Python-бібліотеки, які є типовими для задач NLP: SentenceTransformers, CrossEncoder, text splitters і LanceDB. Винесення цього сервісу в окремий процес дозволяє змінювати модель embeddings або параметри chunking без зміни ASP.NET Core API та React-клієнта. Також це спрощує майбутнє масштабування ML-частини.

Джерелом документів є Google Drive. Це зовнішній сервіс, який зберігає оригінали документів, папки та посилання. Backend отримує документи через Google Drive API, експортує Google Workspace-файли в оброблюваний формат, читає PDF, DOCX, TXT і CSV, а потім зберігає текст і метадані в локальній базі. Оригінальне посилання зберігається, щоб користувач міг перейти до першоджерела.

Типова взаємодія під час пошуку відбувається послідовно. Користувач вводить запит у React. React викликає endpoint ASP.NET Core API. Backend перевіряє JWT і роль користувача, формує запит до FastAPI-сервісу, отримує ranked результати, перетворює їх у відповідний response DTO та повертає клієнту. React будує граф і відкриває модальне вікно фрагмента за дією користувача.

Типова взаємодія під час синхронізації має іншу послідовність. Backend звертається до Google Drive, обходить папки, отримує файли, парсить текст, обчислює hash, оновлює записи SQLite, визначає змінені або видалені

документи, а потім викликає FastAPI endpoint для upsert або delete векторного індексу. Таким чином, API є координатором між джерелом документів і semantic index.

Архітектура з трьома основними частинами має компроміси. Вона складніша, ніж монолітний застосунок, оскільки потребує налаштування HTTP-взаємодії між сервісами. Водночас вона краще відповідає задачі: frontend, backend і vector-search мають різні технологічні вимоги, різні залежності та різні темпи розвитку. Для дипломного проєкту такий поділ є обґрунтованим, оскільки демонструє сучасний підхід до побудови AI-орієнтованих веб-систем.

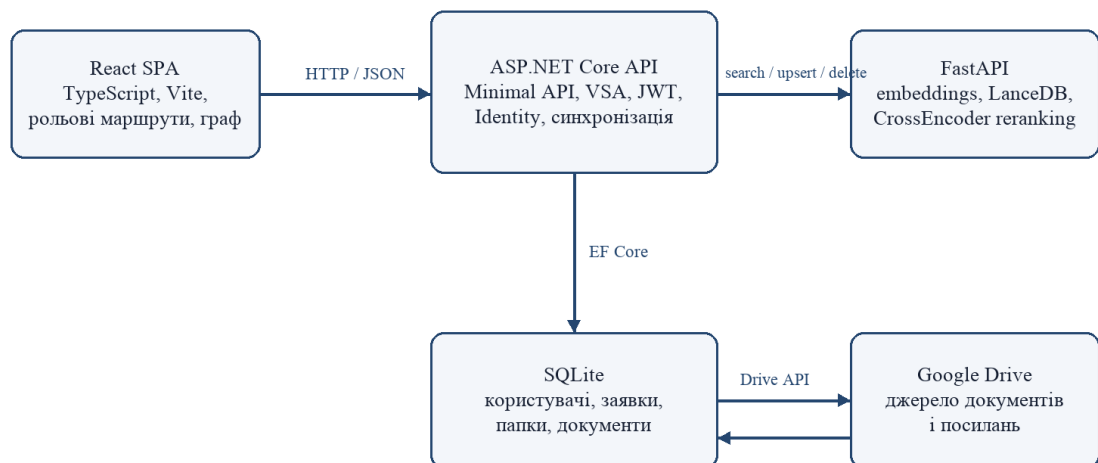


Рисунок 1.1 – Узагальнена архітектура системи EduGraph

### 1.3 Огляд існуючих комплексних рішень

Для вибору підходу зіставлено файловий пошук, повнотекстові системи, керовані векторні сервіси та локальні векторні сховища. Порівняння наведено в таблиці 1.2.

Таблиця 1.2 – Порівняння комплексних рішень пошуку

| Рішення                    | Можливості                              | Розгортання                | Переваги                        | Обмеження для EduGraph                                  |
|----------------------------|-----------------------------------------|----------------------------|---------------------------------|---------------------------------------------------------|
| Google Drive Search        | Назви, метадані, текстовий пошук        | Хмарне                     | Готове джерело документів       | Немає керованого двоетапного semantic retrieval         |
| Elasticsearch / OpenSearch | Повнотекстовий і vector search          | Окремий сервер або кластер | Гнучкі фільтри й індекси        | Надлишкова інфраструктура для локального прототипу      |
| Pinecone / Weaviate Cloud  | Кероване векторне сховище               | Хмарне                     | Масштабування й готовий API     | Зовнішня залежність і потенційні витрати                |
| Qdrant / Milvus            | Векторний пошук і фільтри               | Окремий сервіс             | Розвинені можливості пошуку     | Потрібне додаткове розгортання                          |
| LanceDB                    | Локальне зберігання vectors і метаданих | У процесі FastAPI          | Проста інтеграція з Python [13] | Потрібно окремо вирішувати резервування і масштабування |
| Генеративна RAG-система    | Retrieval і формування відповіді        | Кілька моделей і сервісів  | Зручна синтезована відповідь    | Виходить за межі реалізованого retrieval-прототипу      |

Для поточного масштабу обрано Google Drive як джерело та LanceDB як локальний vector store. Генеративний шар не включено, щоб зберегти прямий зв'язок результату з першоджерелом.

#### 1.4 Аналіз середовищ створення застосунку

Середовища оцінювалися за підтримкою мов, налагодження, пакетних менеджерів і роботи з монорепозиторієм. Результат наведено в таблиці 1.3.

Таблиця 1.3 – Порівняння середовищ розроблення

| Частина            | Розглянуті середовища             | Придатні можливості                           | Прийняте рішення                                  |
|--------------------|-----------------------------------|-----------------------------------------------|---------------------------------------------------|
| ASP.NET Core       | JetBrains Rider; Visual Studio    | C#, NuGet, EF Core, профілі запуску, debugger | Будь-яке з двох; код не залежить від IDE          |
| React / TypeScript | Visual Studio Code; WebStorm      | TSX, ESLint, npm scripts, навігація типів     | VS Code або WebStorm                              |
| FastAPI / Python   | PyCharm; Visual Studio Code       | venv, Python debugger, HTTP-запити            | PyCharm або VS Code                               |
| API                | Scalar; OpenAPI-файл; HTTP-клієнт | Перегляд контрактів і ручні виклики           | OpenAPI/Scalar для ASP.NET; test HTTP для FastAPI |

| Частина | Розглянуті середовища                     | Придатні можливості                    | Прийняте рішення                 |
|---------|-------------------------------------------|----------------------------------------|----------------------------------|
| БД      | EF Core migrations;<br>SQLite CLI/браузер | Еволюція схеми та<br>перевірка таблиць | Міграції як джерело<br>структури |

## 1.5 Огляд ОС, мов програмування та платформ

Варіанти платформ зіставлено за відповідністю фактичним задачам системи. Порівняння наведено в таблиці 1.4.

Таблиця 1.4 – Вибір мов, платформ і сховищ

| Зона               | Альтернативи                              | Обране рішення              | Причина вибору                                                                 |
|--------------------|-------------------------------------------|-----------------------------|--------------------------------------------------------------------------------|
| Основний API       | ASP.NET Core; NestJS;<br>Django           | ASP.NET Core Minimal<br>API | Identity, DI, фонові<br>служби, OpenAPI та<br>типізована доменна<br>логіка [5] |
| Клієнт             | React; Vue; Angular                       | React + TypeScript + Vite   | Фактична компонентна<br>реалізація, хуки й швидке<br>складання SPA [8, 9]      |
| ML/NLP API         | FastAPI; Flask; інтеграція<br>в .NET      | FastAPI                     | Pydantic-контракти та<br>Python-екосистема<br>моделей [11]                     |
| Реляційна БД       | SQLite; PostgreSQL; SQL<br>Server         | SQLite                      | Достатня для локального<br>прототипу, підтримується<br>EF Core provider [6]    |
| Векторне сховище   | LanceDB; Qdrant; Milvus;<br>Pinecone      | LanceDB                     | Локальна робота без<br>окремого search cluster<br>[13]                         |
| Джерело документів | Власне сховище;<br>OneDrive; Google Drive | Google Drive API            | Корпус і структура папок<br>уже існують у<br>зовнішньому сховищі [12]          |

## 1.6 Постановка задачі

Функціональні вимоги згруповано в таблиці 1.5, нефункціональні – у таблиці 1.6.

Таблиця 1.5 – Функціональні вимоги до EduGraph

| ID | Вимога                               | Результат                                                              |
|----|--------------------------------------|------------------------------------------------------------------------|
| F1 | Автентифікація та рольовий<br>доступ | Доступ до маршрутів згідно з<br>Student, Teacher, Admin,<br>SuperAdmin |
| F2 | Подання й оброблення заявки          | Створення Pending-заявки,<br>approve або reject                        |

| ID | Вимога                     | Результат                                                    |
|----|----------------------------|--------------------------------------------------------------|
| F3 | Синхронізація Google Drive | Оновлення папок, документів і локального стану               |
| F4 | Витягування тексту         | DOCX, PDF, TXT, CSV та підтримувані Google Workspace exports |
| F5 | Семантична індексація      | Chunks, embeddings і записи LanceDB                          |
| F6 | Семантичний пошук          | Vector retrieval, фільтрація, reranking, top-k               |
| F7 | Графове подання            | Root, trunk, folder і document вузли                         |
| F8 | Перегляд джерела           | Фрагмент у modal і безпечне посилання Google Drive           |

Таблиця 1.6 – Нефункціональні вимоги до EduGraph

| Категорія        | Вимога                                                      | Критерій                                          |
|------------------|-------------------------------------------------------------|---------------------------------------------------|
| Безпека          | Backend перевіряє JWT і політики незалежно від UI           | Заборонений прямий HTTP-запит відхиляється        |
| Модульність      | Frontend, ASP.NET API і FastAPI розділені                   | Компоненти взаємодіють через HTTP-контракти       |
| Зрозумілість     | Пошук не вимагає знання embeddings                          | Користувач працює із запитом, графом і фрагментом |
| Відтворюваність  | Схема БД визначена migrations, параметри пошуку зафіксовані | Опис відповідає коду [1]                          |
| Розширюваність   | Модель embeddings і СУБД можна замінити                     | Контракти компонентів не залежать від UI          |
| Обмеження якості | Не заявляти кількісну точність без benchmark                | Результати оцінюються функціональними сценаріями  |

## 1.7 Попередній вибір проєктних рішень

Підсумкові рішення та відхилені альтернативи наведено в таблиці 1.7. Вибір відповідає фактичній структурі монорепозиторію [1].

Таблиця 1.7 – Обґрунтування проєктних рішень

| Компонент   | Розглянуті варіанти                        | Обрано                        | Обґрунтування                            |
|-------------|--------------------------------------------|-------------------------------|------------------------------------------|
| Архітектура | Моноліт; два компоненти; окремий ML-сервіс | React + ASP.NET API + FastAPI | Розділення UI, доменної логіки та NLP    |
| API         | Controllers; Minimal API                   | Minimal API + VSA             | Локалізація request, validator і handler |

| Компонент    | Розглянуті варіанти                         | Обрано               | Обґрунтування                                                |
|--------------|---------------------------------------------|----------------------|--------------------------------------------------------------|
| Авторизація  | Власні таблиці; Identity                    | Identity + JWT       | Користувачі, ролі й стандартне хешування паролів [7, 17]     |
| Пошук        | Keyword; embeddings; embeddings + reranking | Двоетапний retrieval | Швидкий vector candidate search і уточнення порядку [14, 15] |
| Граф         | Список; SVG; Canvas force graph             | react-force-graph-2d | Canvas-рендеринг і кероване розміщення вузлів [10]           |
| API-контракт | Неформальні DTO; OpenAPI                    | OpenAPI              | Машиночитаний опис endpoint-ів [18]                          |



## 2 ФУНКЦІОНАЛЬНА ЧАСТИНА

### 2.1 Проектування варіантів використання системи

Функціональна модель визначає акторів і доступні їм сценарії без опису внутрішньої реалізації. Акторів наведено в таблиці 2.1.

Таблиця 2.1 – Актори системи EduGraph

| Актор                      | Основні дії                                       | Обмеження                  |
|----------------------------|---------------------------------------------------|----------------------------|
| Неавторизований користувач | Вхід, подання заявки                              | Немає доступу до корпусу   |
| Student                    | Пошук, граф, фрагмент, оригінал                   | Немає адміністративних дій |
| Teacher                    | Пошук, заявки, студенти, викладачі, синхронізація | Не керує адміністраторами  |
| Admin                      | Пошук, заявки, студенти, викладачі, синхронізація | Не керує адміністраторами  |
| SuperAdmin                 | Усі сценарії, зокрема адміністратори              | Найширші права             |

Матриця доступу наведена в таблиці 2.2. Остаточна перевірка ролей виконується ASP.NET Core API, а не клієнтським маршрутом.

Таблиця 2.2 – Матриця доступу до сценаріїв

| Сценарій                   | Student | Teacher | Admin | SuperAdmin |
|----------------------------|---------|---------|-------|------------|
| Семантичний пошук          | +       | +       | +     | +          |
| Перегляд графу й документа | +       | +       | +     | +          |
| Опрацювання заявок         | –       | +       | +     | +          |
| Керування студентами       | –       | +       | +     | +          |
| Керування викладачами      | –       | +       | +     | +          |
| Керування адміністраторами | –       | –       | –     | +          |
| Запуск синхронізації       | –       | +       | +     | +          |

Зв'язки акторів із варіантами використання показано стрілками на рисунку 2.1.

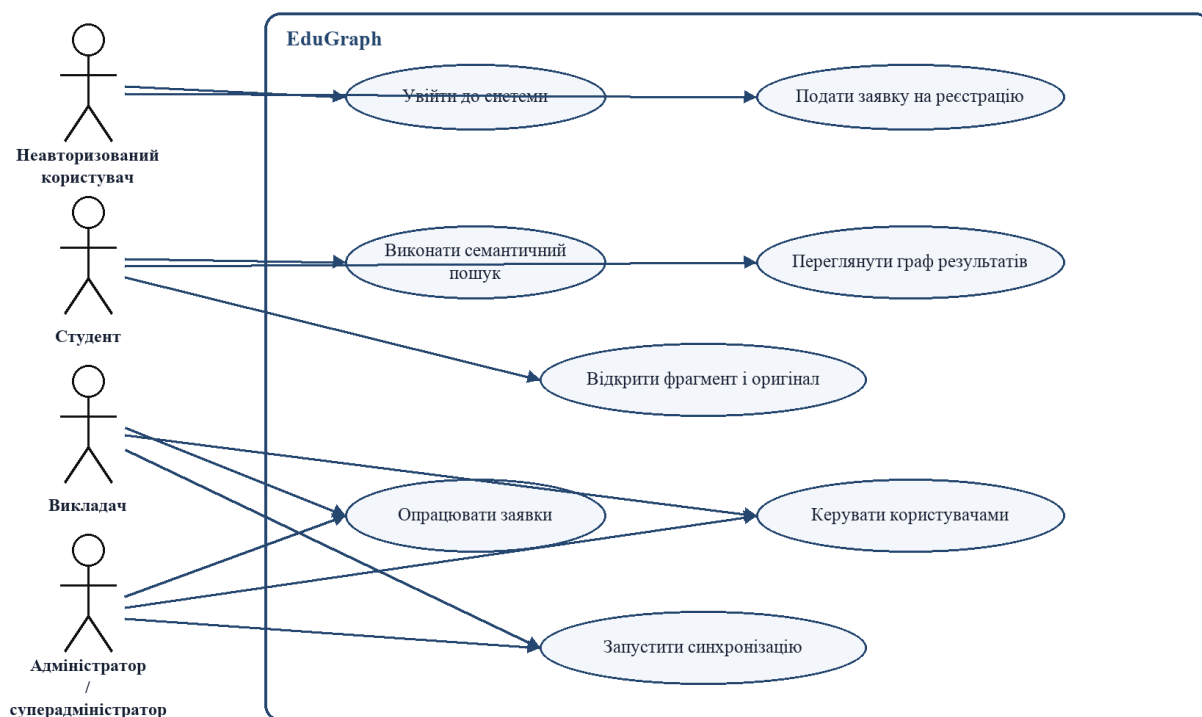


Рисунок 2.1 – Загальна діаграма варіантів використання EduGraph

## 2.2 Специфікації основних сценаріїв

Для перевірки функціональних вимог визначено шість основних сценаріїв. Їх перелік наведено в таблиці 2.3, а повні специфікації з передумовами, основним потоком, альтернативами й постумовами винесено до додатка А.

Таблиця 2.3 – Перелік основних сценаріїв

| Код   | Сценарій                        | Актор                                        |
|-------|---------------------------------|----------------------------------------------|
| UC-01 | Увійти до системи               | Користувач                                   |
| UC-02 | Подати заявку на реєстрацію     | Неавторизований користувач                   |
| UC-03 | Опрацювати заявку               | Teacher, Admin, SuperAdmin                   |
| UC-04 | Виконати семантичний пошук      | Авторизований користувач                     |
| UC-05 | Переглянути фрагмент і оригінал | Авторизований користувач                     |
| UC-06 | Запустити синхронізацію         | Teacher, Admin, SuperAdmin або фонові служба |

### 2.3 Діаграми сценаріїв та взаємодії

Заявка має три збережені стани. Перехід до Approved супроводжується створенням користувача, а Rejected не створює облікового запису. Діаграму станів наведено на рисунку 2.2.

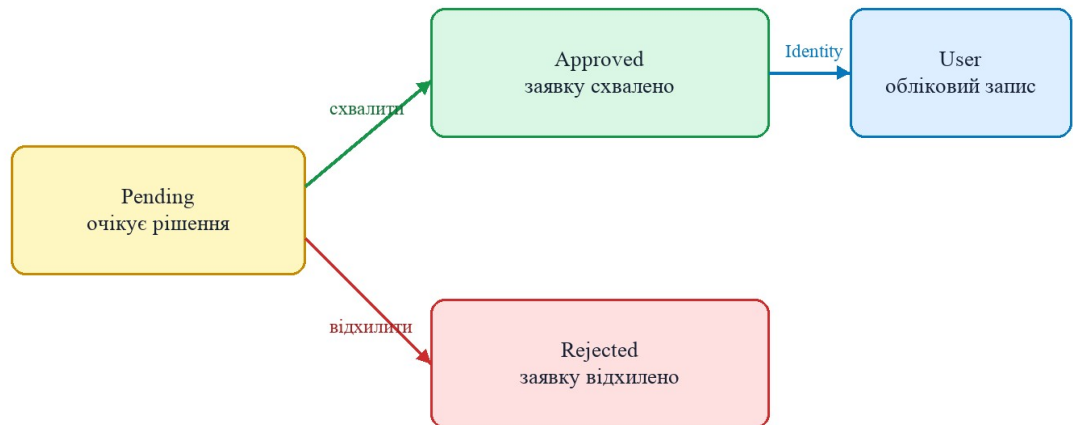


Рисунок 2.2 – Діаграма станів заявки на реєстрацію

## 3 ПРОЄКТНА ЧАСТИНА

### 3.1 Вибір базових технологій, платформ і стандартів

Аналітична частина обґрунтовує вибір, тому тут зафіксовано тільки фактичний стек і його роль у реалізації. Версії наведено в таблиці 3.1 [1].

Таблиця 3.1 – Фактичний технологічний стек EduGraph

| Компонент       | Технології та версії                                            | Призначення                                          |
|-----------------|-----------------------------------------------------------------|------------------------------------------------------|
| Frontend        | React 19.2.0; TypeScript 5.9.3; Vite 7.3.3; React Router 7.15.1 | SPA, рольові маршрути й інтерфейс [8, 9]             |
| Граф            | react-force-graph-2d 1.29.1                                     | Canvas-візуалізація вузлів і зв'язків [10]           |
| Backend         | .NET 9; ASP.NET Core Minimal API; JWT Bearer 9.0.11             | HTTP API, політики, фонові служби [5]                |
| Дані            | EF Core SQLite 9.0.11; Identity 9.0.11                          | Реляційний стан, користувачі й ролі [6, 7]           |
| Google Drive    | Google Drive API 1.72.0.3970; OpenXML 3.3.0; PdfPig 0.1.12      | Синхронізація та витягування тексту [12, 19, 20]     |
| Semantic search | FastAPI; SentenceTransformers; LanceDB; CrossEncoder            | Індексація, vector retrieval і reranking [11, 13–15] |

### 3.2 Визначення функціональних залежностей

Залежності визначають необхідні взаємодії компонентів, а не відношення успадкування чи фізичні зв'язки таблиць. Їх наведено в таблиці 3.2.

Таблиця 3.2 – Функціональні залежності компонентів

| Компонент          | Залежить від                        | Наслідок недоступності                                    |
|--------------------|-------------------------------------|-----------------------------------------------------------|
| React-клієнт       | ASP.NET Core API                    | Не виконує авторизацію, пошук і синхронізацію             |
| Реалізація бекенду | SQLite                              | Не зберігає користувачів, заявки й метадані               |
| Реалізація бекенду | Google Drive API                    | Не оновлює корпус, але може працювати з наявним індексом  |
| Реалізація бекенду | FastAPI                             | Не виконує semantic search та upsert/delete vector chunks |
| FastAPI-сервіс     | SentenceTransformers і CrossEncoder | Не створює embeddings і не уточнює порядок                |
| FastAPI-сервіс     | LanceDB                             | Не зберігає й не шукає vector chunks                      |

### 3.3 Проектування бази даних

#### 3.3.1 Вибір СУБД

SQLite відповідає масштабу локального прототипу й використовується через офіційний EF Core provider [6]. Файли зберігаються в Google Drive, embeddings – у LanceDB, тому реляційна БД містить користувачів, заявки та метадані.

#### 3.3.2 Опис логічної структури БД

Логічна структура містить прикладні сутності User, SignUpApplication, UniversityDocument і UniversityFolder. Між ними не визначено EF Core зовнішніх ключів або navigation properties. Папка документа зберігається денормалізовано в FolderName, а зв'язок із LanceDB є логічним через GoogleDriveId/document\_id [1]. Схему наведено на рисунку 3.1.

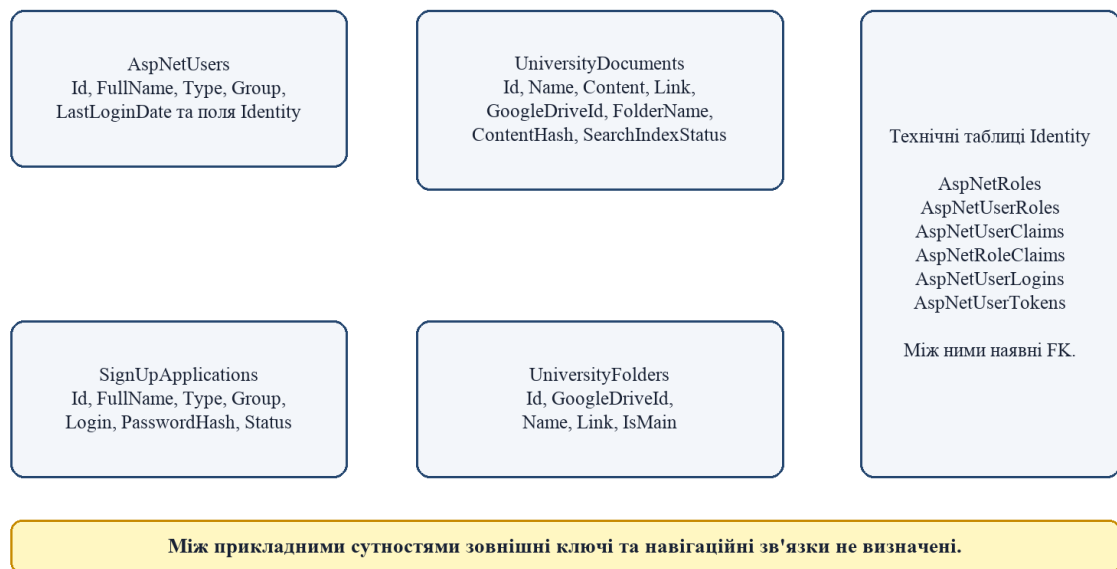


Рисунок 3.1 – Логічна структура даних EduGraph

#### 3.3.3 Опис фізичної структури БД

Фізична схема на рисунку 3.2 підтверджує відсутність зв'язків між прикладними таблицями. Стрілки на схемі належать тільки технічним

таблицям ASP.NET Identity, де FK пов’язують користувачів, ролі, claims, logins і tokens.

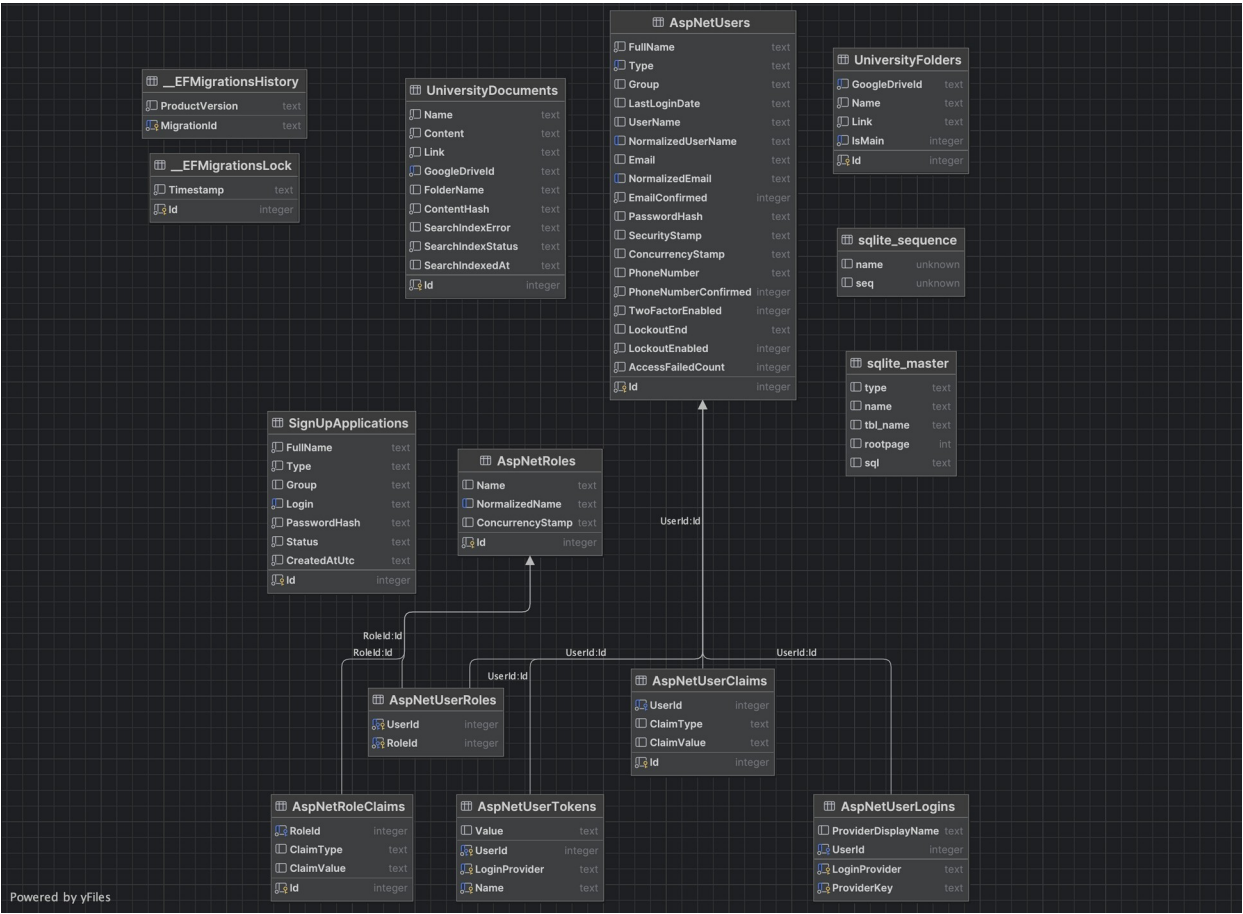


Рисунок 3.2 – Фізична схема SQLite бази даних EduGraph

Поля всіх прикладних, Identity та службових таблиць наведено в таблицях 3.3–3.14. Тип TEXT використовується також для дат і enum-значень відповідно до поточного EF Core mapping.

Таблиця 3.3 – Поля таблиці SignUpApplications

| Поле         | Тип SQLite | NULL | Ключ / індекс     | Призначення                    |
|--------------|------------|------|-------------------|--------------------------------|
| Id           | INTEGER    | ні   | PK, autoincrement | Ідентифікатор заявки           |
| CreatedAtUtc | TEXT       | ні   | –                 | Дата й час створення UTC       |
| FullName     | TEXT       | ні   | –                 | ПІБ заявника                   |
| Group        | TEXT       | так  | –                 | Навчальна група                |
| Login        | TEXT       | ні   | UNIQUE            | Логін майбутнього користувача  |
| PasswordHash | TEXT       | ні   | –                 | Хеш пароля                     |
| Status       | TEXT       | ні   | –                 | Pending, Approved або Rejected |

| Поле | Тип SQLite | NULL | Ключ / індекс | Призначення                 |
|------|------------|------|---------------|-----------------------------|
| Type | TEXT       | ні   | –             | Тип майбутнього користувача |

Таблиця 3.4 – Поля таблиці UniversityDocuments

| Поле              | Тип SQLite | NULL | Ключ / індекс     | Призначення                        |
|-------------------|------------|------|-------------------|------------------------------------|
| Id                | INTEGER    | ні   | PK, autoincrement | Локальний ідентифікатор            |
| Content           | TEXT       | ні   | –                 | Витягнутий лінійний текст          |
| ContentHash       | TEXT       | ні   | –                 | SHA-256 тексту                     |
| FolderName        | TEXT       | так  | –                 | Денормалізована назва папки        |
| GoogleDriveId     | TEXT       | ні   | UNIQUE            | Ідентифікатор файла Google Drive   |
| Link              | TEXT       | ні   | –                 | Посилання на оригінал              |
| Name              | TEXT       | ні   | –                 | Назва документа                    |
| SearchIndexError  | TEXT       | так  | –                 | Текст останньої помилки індексації |
| SearchIndexStatus | TEXT       | ні   | –                 | Стан індексації                    |
| SearchIndexedAt   | TEXT       | так  | –                 | Дата останньої позначки Indexed    |

Таблиця 3.5 – Поля таблиці UniversityFolders

| Поле          | Тип SQLite | NULL | Ключ / індекс         | Призначення                             |
|---------------|------------|------|-----------------------|-----------------------------------------|
| Id            | INTEGER    | ні   | PK, autoincrement     | Локальний ідентифікатор                 |
| GoogleDriveId | TEXT       | ні   | UNIQUE                | Ідентифікатор папки або службового root |
| IsMain        | INTEGER    | ні   | UNIQUE WHERE IsMain=1 | Ознака головного вузла                  |
| Link          | TEXT       | ні   | –                     | Посилання Google Drive                  |
| Name          | TEXT       | ні   | –                     | Назва папки                             |

Таблиця 3.6 – Поля таблиці AspNetUsers

| Поле     | Тип SQLite | NULL | Ключ / індекс | Призначення               |
|----------|------------|------|---------------|---------------------------|
| Id       | INTEGER    | ні   | PK            | Ідентифікатор користувача |
| UserName | TEXT       | так  | –             | Ім'я входу                |

| Поле                 | Тип SQLite | NULL | Ключ / індекс | Призначення                         |
|----------------------|------------|------|---------------|-------------------------------------|
| NormalizedUserName   | TEXT       | так  | UNIQUE index  | Нормалізоване ім'я входу            |
| Email                | TEXT       | так  | index         | Електронна пошта                    |
| NormalizedEmail      | TEXT       | так  | index         | Нормалізована пошта                 |
| EmailConfirmed       | INTEGER    | ні   | –             | Ознака підтвердження пошти          |
| PasswordHash         | TEXT       | так  | –             | Хеш пароля Identity                 |
| SecurityStamp        | TEXT       | так  | –             | Маркер безпеки                      |
| ConcurrencyStamp     | TEXT       | так  | –             | Маркер конкурентних змін            |
| PhoneNumber          | TEXT       | так  | –             | Номер телефону                      |
| PhoneNumberConfirmed | INTEGER    | ні   | –             | Підтвердження телефону              |
| TwoFactorEnabled     | INTEGER    | ні   | –             | Ознака 2FA                          |
| LockoutEnd           | TEXT       | так  | –             | Завершення блокування               |
| LockoutEnabled       | INTEGER    | ні   | –             | Дозвіл блокування                   |
| AccessFailedCount    | INTEGER    | ні   | –             | Кількість невдалих входів           |
| FullName             | TEXT       | ні   | –             | ПІБ користувача                     |
| Type                 | TEXT       | ні   | index         | Student, Teacher, Admin, SuperAdmin |
| Group                | TEXT       | так  | –             | Навчальна група                     |
| LastLoginDate        | TEXT       | так  | –             | Дата останнього входу               |

Таблиця 3.7 – Поля таблиці AspNetRoles

| Поле             | Тип SQLite | NULL | Ключ / індекс | Призначення              |
|------------------|------------|------|---------------|--------------------------|
| Id               | INTEGER    | ні   | PK            | Ідентифікатор ролі       |
| Name             | TEXT       | так  | –             | Назва ролі               |
| NormalizedName   | TEXT       | так  | UNIQUE index  | Нормалізована назва      |
| ConcurrencyStamp | TEXT       | так  | –             | Маркер конкурентних змін |

Таблиця 3.8 – Поля таблиці AspNetRoleClaims

| Поле   | Тип SQLite | NULL | Ключ / індекс    | Призначення         |
|--------|------------|------|------------------|---------------------|
| Id     | INTEGER    | ні   | PK               | Ідентифікатор claim |
| RoleId | INTEGER    | ні   | FK → AspNetRoles | Роль                |



| Поле       | Тип SQLite | NULL | Ключ / індекс | Призначення    |
|------------|------------|------|---------------|----------------|
| ClaimType  | TEXT       | так  | –             | Тип claim      |
| ClaimValue | TEXT       | так  | –             | Значення claim |

Таблиця 3.9 – Поля таблиці AspNetUserClaims

| Поле       | Тип SQLite | NULL | Ключ / індекс    | Призначення         |
|------------|------------|------|------------------|---------------------|
| Id         | INTEGER    | ні   | PK               | Ідентифікатор claim |
| UserId     | INTEGER    | ні   | FK → AspNetUsers | Користувач          |
| ClaimType  | TEXT       | так  | –                | Тип claim           |
| ClaimValue | TEXT       | так  | –                | Значення claim      |

Таблиця 3.10 – Поля таблиці AspNetUserLogins

| Поле                | Тип SQLite | NULL | Ключ / індекс    | Призначення       |
|---------------------|------------|------|------------------|-------------------|
| LoginProvider       | TEXT       | ні   | PK (частина)     | Провайдер входу   |
| ProviderKey         | TEXT       | ні   | PK (частина)     | Ключ у провайдера |
| ProviderDisplayName | TEXT       | так  | –                | Назва провайдера  |
| UserId              | INTEGER    | ні   | FK → AspNetUsers | Користувач        |

Таблиця 3.11 – Поля таблиці AspNetUserRoles

| Поле   | Тип SQLite | NULL | Ключ / індекс        | Призначення |
|--------|------------|------|----------------------|-------------|
| UserId | INTEGER    | ні   | PK, FK → AspNetUsers | Користувач  |
| RoleId | INTEGER    | ні   | PK, FK → AspNetRoles | Роль        |

Таблиця 3.12 – Поля таблиці AspNetUserTokens

| Поле          | Тип SQLite | NULL | Ключ / індекс        | Призначення     |
|---------------|------------|------|----------------------|-----------------|
| UserId        | INTEGER    | ні   | PK, FK → AspNetUsers | Користувач      |
| LoginProvider | TEXT       | ні   | PK (частина)         | Провайдер       |
| Name          | TEXT       | ні   | PK (частина)         | Назва токена    |
| Value         | TEXT       | так  | –                    | Значення токена |

Таблиця 3.13 – Поля таблиці \_\_EFMigrationsHistory

| Поле           | Тип SQLite | NULL | Ключ / індекс | Призначення                         |
|----------------|------------|------|---------------|-------------------------------------|
| MigrationId    | TEXT       | ні   | РК            | Ідентифікатор застосованої міграції |
| ProductVersion | TEXT       | ні   | –             | Версія EF Core                      |

Таблиця 3.14 – Поля таблиці \_\_EFMigrationsLock

| Поле      | Тип SQLite | NULL | Ключ / індекс | Призначення              |
|-----------|------------|------|---------------|--------------------------|
| Id        | INTEGER    | ні   | РК            | Ідентифікатор блокування |
| Timestamp | TEXT       | ні   | –             | Час блокування міграції  |

### 3.3.4 Підтримка цілісності даних

Цілісність прикладних сутностей підтримують фабричні й доменні методи, NOT NULL, унікальні індекси GoogleDriveId і Login та стандартні FK Identity. Окремої транзакційної гарантії між SQLite і LanceDB немає: це два сховища, узгодження яких виконує сервіс синхронізації [1].

## 3.4 Реалізація підсистеми семантичного пошуку

Підсистема є retrieval-сервісом, а не генеративною чат-LLM. FastAPI приймає документи й запити, SentenceTransformers створює embeddings, LanceDB виконує cosine retrieval, а CrossEncoder уточнює порядок кандидатів [3, 4, 11, 13–15].

### 3.4.1 HTTP-контракти та моделі

Таблиця 3.15 – Контракти FastAPI-сервісу

| Метод і маршрут        | Модель запиту | Результат                             |
|------------------------|---------------|---------------------------------------|
| GET /health            | –             | {"status": "ok"}                      |
| POST /documents/upsert | UpsertRequest | inserted_chunks,<br>skipped_documents |
| POST /documents/delete | DeleteRequest | deleted_documents                     |
| POST /search           | SearchRequest | Масив SearchResult                    |

Таблиця 3.16 – Параметри семантичного пошуку

| Параметр             | Фактичне значення                          | Призначення                    |
|----------------------|--------------------------------------------|--------------------------------|
| Embedding model      | intfloat/multilingual-e5-base              | Вектори документів і запиту    |
| Reranker             | cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 | Повторне ранжування            |
| chunk_size / overlap | 1000 / 150 символів                        | Поділ великих документів [16]  |
| Embedding dimension  | 768                                        | Розмір vector у LanceDB        |
| Embedding batch      | 8                                          | Пакетне кодування chunks       |
| top_k                | 5, максимум 50                             | Кількість фінальних документів |
| Candidate count      | $\max(\text{top\_k} \times 8, 30)$         | Кандидати до reranking         |
| min_score            | 0,75                                       | Попіг cosine score             |
| min_rerank_score     | -1,0                                       | Попіг raw CrossEncoder score   |

### 3.4.2 Підготовка тексту та індексація

Перед поділом стискаються послідовності пробілів і табуляцій. RecursiveCharacterTextSplitter ділить текст за абзацами, рядками, розділовими знаками, пробілами й, за потреби, посимвольно [16]. Для chunks використовується префікс passage:, для запиту – query:. Нормалізовані embeddings мають розмірність 768 [4].

### 3.4.3 Фізична структура LanceDB

Таблиця 3.17 – Поля таблиці LanceDB document\_chunks

| Поле         | Тип        | Призначення                                               |
|--------------|------------|-----------------------------------------------------------|
| document_id  | string     | Google Drive ID документа                                 |
| chunk_id     | string     | Ідентифікатор document_id:index                           |
| chunk_index  | int64      | Порядковий номер фрагмента                                |
| title        | string     | Назва документа                                           |
| content      | string     | Текст фрагмента                                           |
| url          | string     | Посилання на оригінал                                     |
| folder_name  | string     | Назва папки                                               |
| content_hash | string     | Хеш документа; не використовується FastAPI для порівняння |
| vector       | float[768] | Нормалізований embedding                                  |

### 3.4.4 Алгоритм пошуку та reranking

Послідовність індексації та пошуку наведено на рисунку 3.3. Після cosine retrieval результати нижче min\_score відкидаються, пари query і title+content оцінюються CrossEncoder, сортуються за raw score, після чого залишається один найкращий chunk на документ.

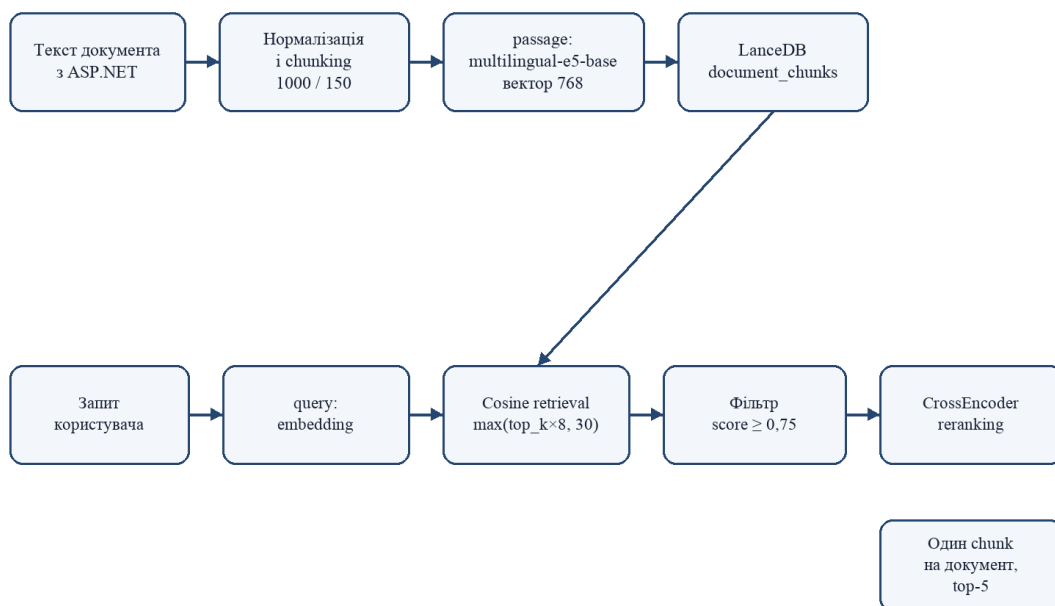


Рисунок 3.3 – Алгоритм індексації та семантичного пошуку

### 3.4.5 Обмеження поточної реалізації

Upsert видаляє старі chunks до побудови нових і не є атомарним. Окремий ANN-індекс у коді не створюється. Result помилки upsert/delete на рівні синхронізації не перевіряється перед зміною локального стану, тому текст не стверджує гарантовану узгодженість SQLite і LanceDB. Кількісна перевага reranking також не заявляється без benchmark-набору [1].

### 3.5 Модель графового подання результатів пошуку

Граф є runtime-моделлю React-клієнта, а не графовою БД, онтологією або алгоритмом кластеризації. ForceGraph2D відображає дані через Canvas, а підготовку вузлів, фіксовані координати та hit-testing виконує код сторінки [1, 8, 10].

### 3.5.1 Формування вузлів і зв'язків

Таблиця 3.18 – Вузли графового подання

| Тип      | Створення                        | Поля                              | Зв'язки                                      |
|----------|----------------------------------|-----------------------------------|----------------------------------------------|
| root     | Один вузол root:g7               | id, title, type                   | Початок trunk-ланцюга; fallback для document |
| trunk    | По одному на folder і trunk:tail | id, title, type                   | Послідовний невидимий ланцюг                 |
| folder   | Із FolderResponse                | id, title, url, type              | Зв'язок від відповідного trunk               |
| document | Із SearchDocumentResponse        | id, title, url, content, parentId | До folder за точним folderName або до root   |

Фактичну структуру зв'язків показано на рисунку 3.4. Поле `chunkId` використовується у складі `id` вузла документа; `score` і `chunkIndex` у `GraphNode` не зберігаються.

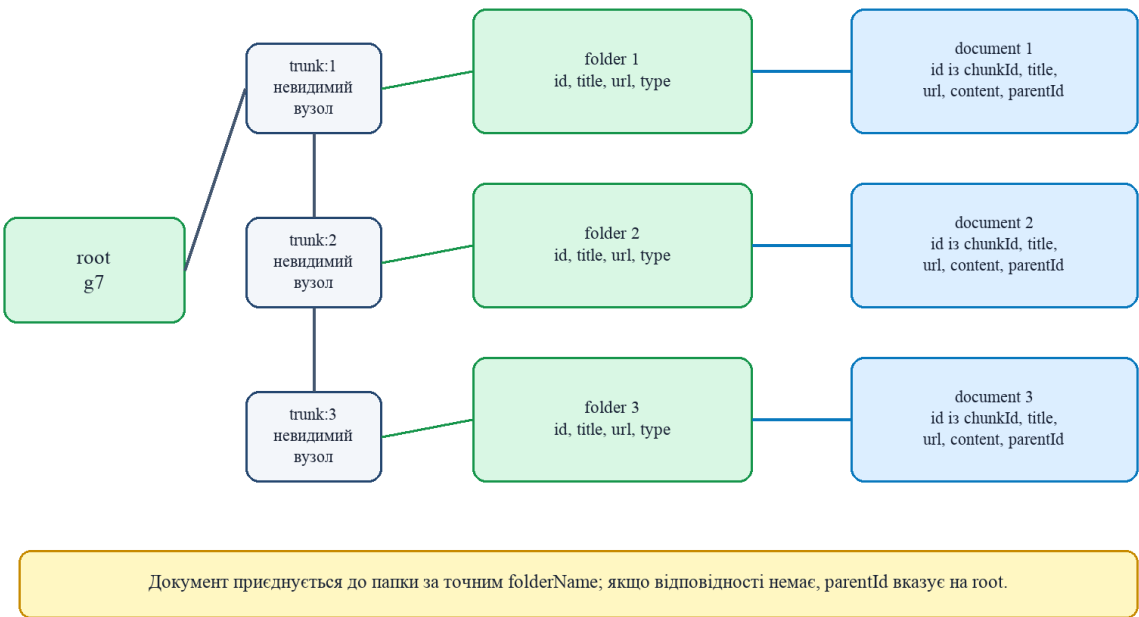


Рисунок 3.4 – Модель побудови графу на клієнті

### 3.5.2 Адаптивне розміщення і Canvas-рендеринг

`applyGraphLayout` обчислює `x`, `y`, `fx` і `fy` для трьох діапазонів ширини: менше 768 px, 768–1279 px і від 1280 px. Папки чергуються зліва та справа від trunk-осі, документи групуються біля parent. `paintNode` малює вузли через `CanvasRenderingContext2D`, а `paintPointerArea` визначає клікабельну область.

React-стан і перерахунок використовують `useState`, `useEffect`, `useMemo`, `useCallback` і `useRef` [8].

### 3.5.3 Взаємодія з результатом

Клік по folder відкриває перевірене посилання Google Drive. Клік по document передає `title`, `folderName`, `content` і `url` до `DocumentModal`. Modal закривається кнопкою, `backdrop` або `Escape`. Лінії графа задають `source` і `target`, але в поточному `ForceGraph2D` не мають візуальних `arrowheads`.

## 3.6 Проєктування серверної частини та синхронізації

ASP.NET Core API є координатором авторизації, реляційного стану, Google Drive і FastAPI. Endpoint-и організовані вертикальними функціональними зрізами; повний каталог маршрутів винесено до додатка Б [1, 5].

### 3.6.1 Маршрути API та політики доступу

Таблиця 3.19 – Групи маршрутів і доступ

| Група                                  | Доступ                     | Призначення                         |
|----------------------------------------|----------------------------|-------------------------------------|
| auth                                   | Анонімний                  | login і створення заявки            |
| google-drive/folders, search-documents | Будь-яка авторизована роль | Структура папок і пошук             |
| google-drive/root-folder-link, sync    | Teacher, Admin, SuperAdmin | Посилання й постановка sync у чергу |
| sign-up-applications                   | Teacher, Admin, SuperAdmin | Перегляд, approve, reject           |
| students, teachers                     | Teacher, Admin, SuperAdmin | Керування користувачами             |
| admins                                 | SuperAdmin                 | Керування адміністраторами          |

### 3.6.2 Реєстрація та заявки

Signup створює `SignUpApplication`, а не користувача. Approve створює `Identity User` і призначає роль; reject змінює статус без створення облікового запису. JWT містить `role claim` відповідно до RFC 7519 [7, 17].

### 3.6.3 Алгоритм синхронізації Google Drive

Синхронізацію запускає фонові служба одразу після старту й далі після чотиригодинної паузи або ручний endpoint через bounded-чергу місткістю один. Спільний semaphore не допускає паралельних запусків. Алгоритм показано на рисунку 3.5 [1, 12].

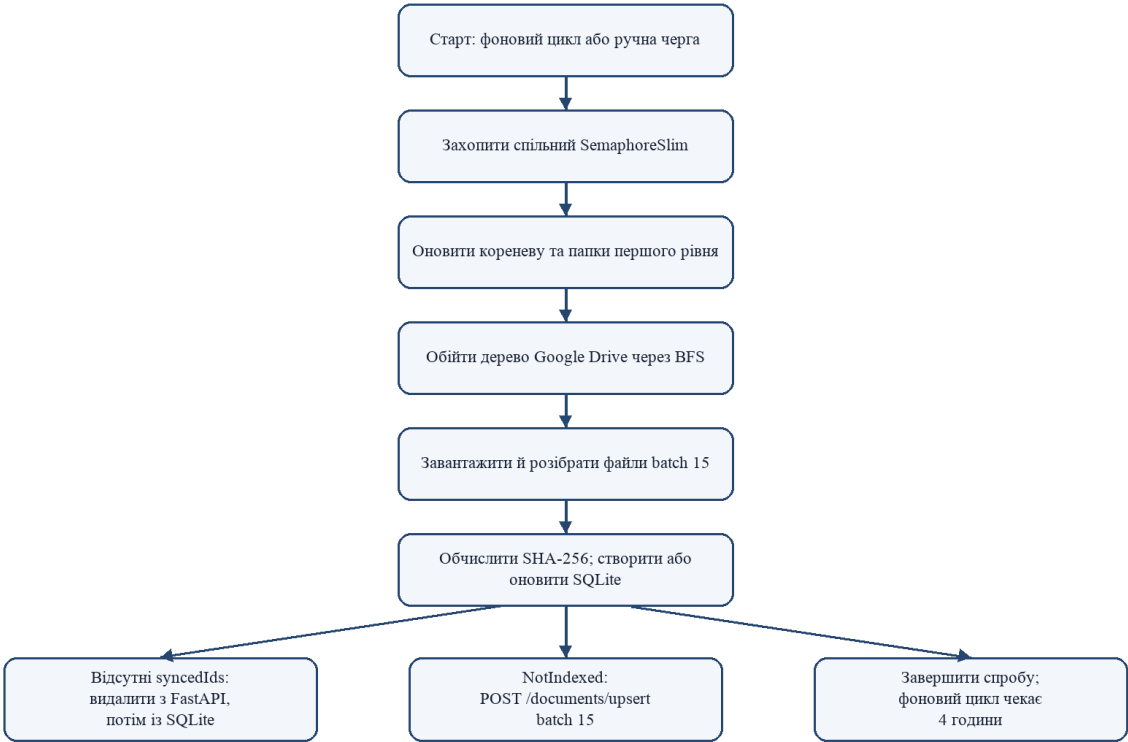


Рисунок 3.5 – Блок-схема алгоритму синхронізації Google Drive

### 3.6.4 Парсинг документів і обмеження форматів

Таблиця 3.20 – Підтримувані формати та обмеження парсингу

| Джерело / MIME | Експорт | Обробка                            | Обмеження                                                      |
|----------------|---------|------------------------------------|----------------------------------------------------------------|
| DOCX           | –       | Open XML Body.InnerText [19]       | Втрачаються зображення, структура й частина складних елементів |
| PDF            | –       | PdfPig page.Text [20]              | OCR відсутній; скан без text layer дає порожній текст          |
| TXT            | –       | StreamReader UTF-8 з BOM detection | Інші кодування можуть бути прочитані некоректно                |
| CSV            | –       | Читання як звичайного тексту       | Стовпці не розбираються окремо                                 |
| Google Docs    | DOCX    | Далі Open XML                      | Залежить від якості export                                     |

| Джерело / MIME | Експорт | Обробка               | Обмеження                        |
|----------------|---------|-----------------------|----------------------------------|
| Google Sheets  | CSV     | Далі текстове читання | Структура аркушів не моделюється |
| Google Slides  | PPTX    | Парсер відсутній      | Фактично не підтримується        |

### 3.6.5 Життєвий цикл індексації документа

Новий або оновлений UniversityDocument переходить у NotIndexed. Такі записи надсилаються до FastAPI batch-ами по 15 і після виклику позначаються Indexed. Поточний Update не порівнює hash перед MarkAsNotIndexed, тому успішно прочитані документи можуть переіндексуватися після кожної синхронізації. Failed не є гарантованим результатом HTTP-помилки через неперевірений Result [1].

### 3.7 Структура та екрани інтерфейсу користувача

Проектна частина фіксує реалізовані екрани, а не повторює функціональні специфікації. Сторінку входу, подання заявки та оброблення заявок перенесено сюди з функціональної частини.

Відповідний стан інтерфейсу наведено на рисунок 3.6.

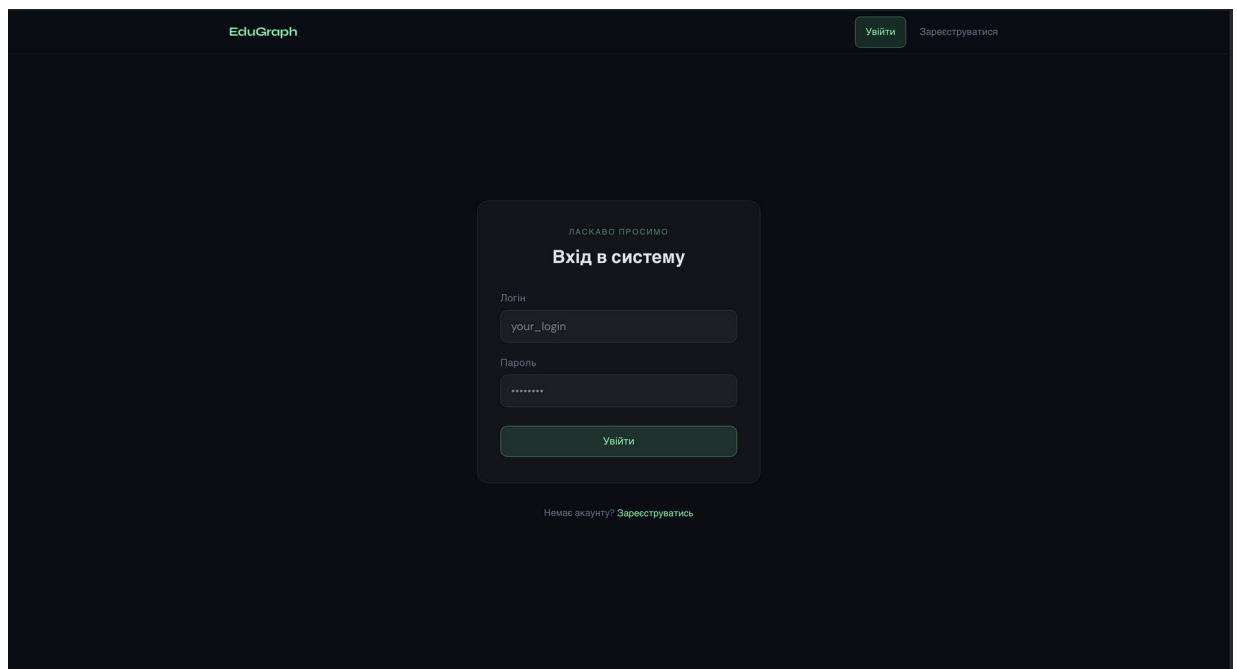


Рисунок 3.6 – Сторінка входу користувача в EduGraph

Відповідний стан інтерфейсу наведено на рисунок 3.7.



ЕдуГраф

Увійти Зареєструватися

НОВИЙ АКАУНТ

**Заявка на реєстрацію**

Логін

your\_login

ПІБ

Іванов Іван Іванович

Пароль

\*\*\*\*\*

Підтвердіть пароль

\*\*\*\*\*

Роль

☒ Студент ☐ Викладач

Група

Ю-21

Надіслати заявку

Рисунок 3.7 – Подання заявки на реєстрацію студентом

Відповідний стан інтерфейсу наведено на рисунок 3.8.

ЕдуГраф

Синхронізувати

Пошук Заявки Студенти

Адміністрування

**Заявки на реєстрацію**

| ID | ПІБ                  | ТИП     | ГРУПА   | ДІЇ                                      |
|----|----------------------|---------|---------|------------------------------------------|
| 3  | Іванов Іван Іванович | Студент | КІ-22-1 | <div>Схвалити</div> <div>Відхилити</div> |

← Попередня 1/1 Наступна →

Рисунок 3.8 – Перегляд і оброблення заявок на реєстрацію

Відповідний стан інтерфейсу наведено на рисунок 3.9.

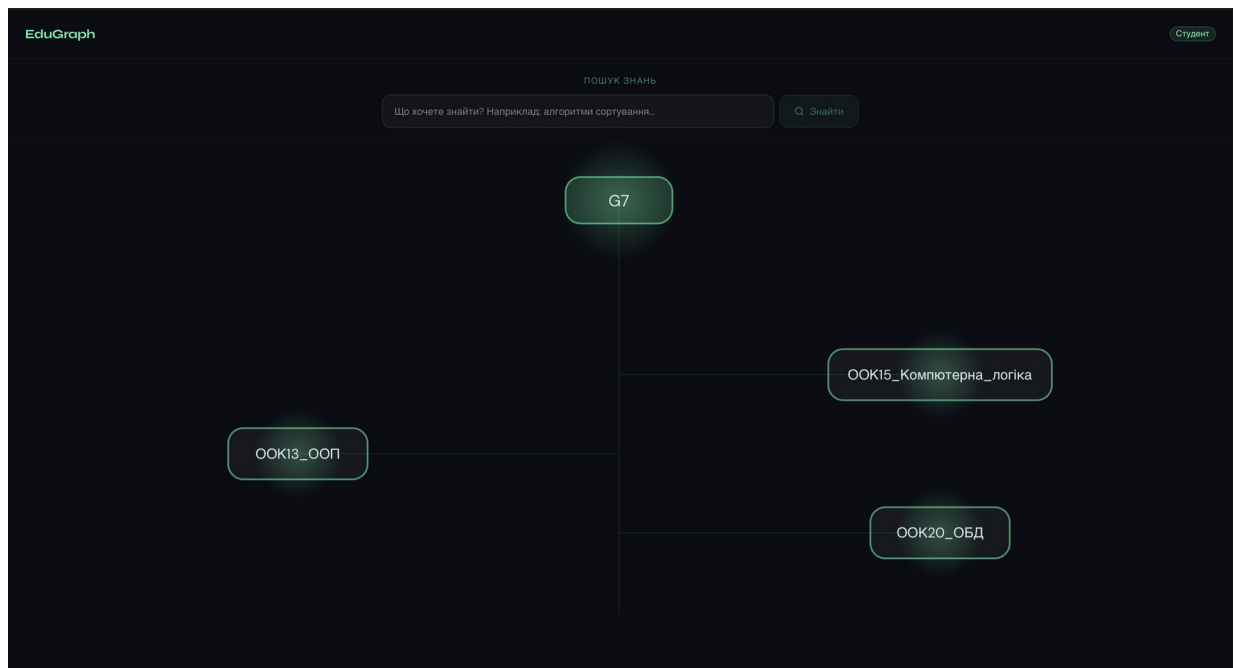


Рисунок 3.9 – Початковий стан графової навігації

Відповідний стан інтерфейсу наведено на рисунок 3.10.

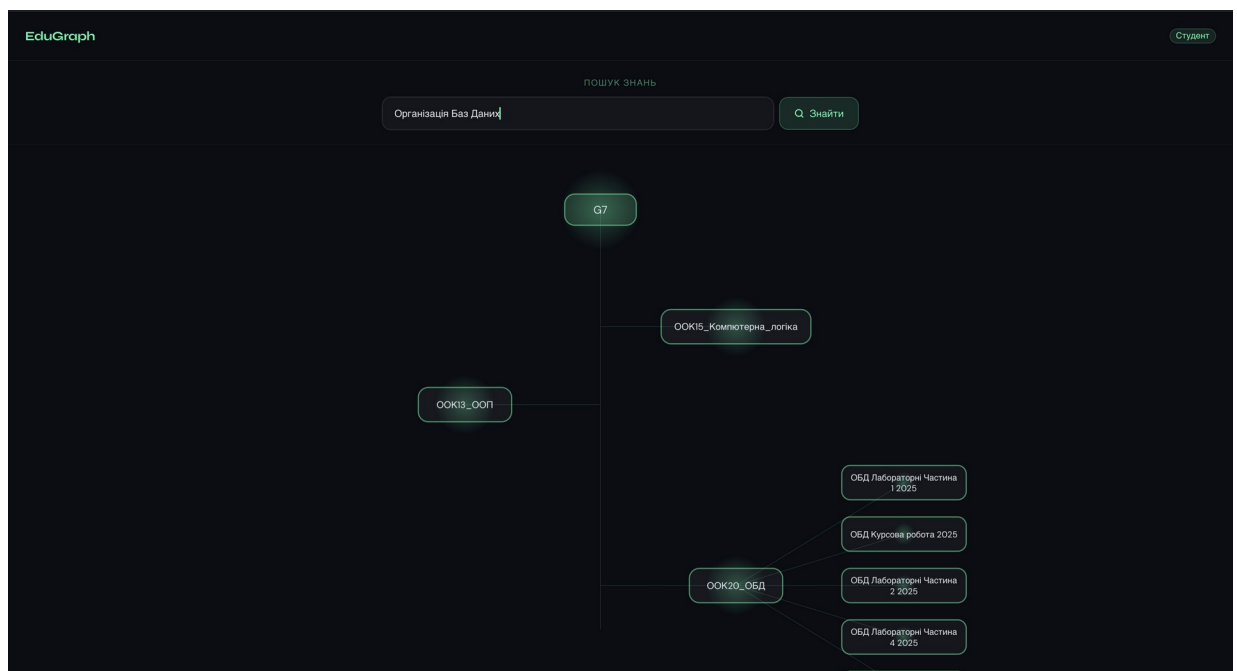


Рисунок 3.10 – Граф результатів семантичного пошуку

Відповідний стан інтерфейсу наведено на рисунок 3.11.

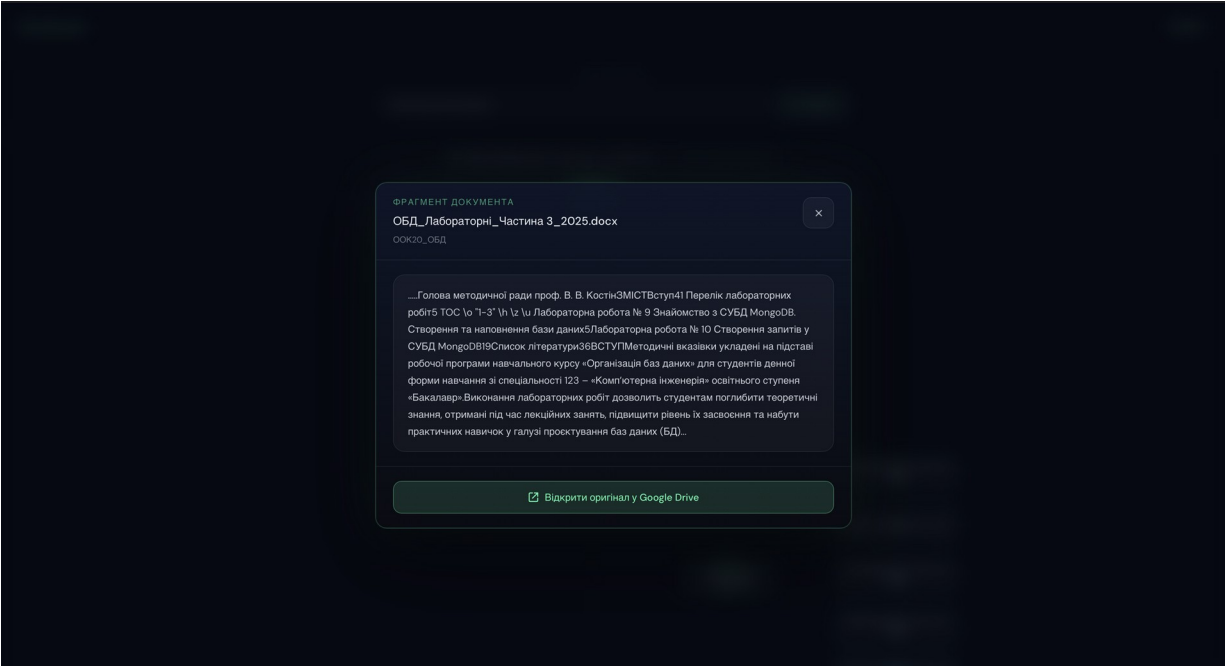


Рисунок 3.11 – Перегляд релевантного фрагмента документа

3.8 Експериментальне тестування застосунку

Перевірка має функціональний характер; кількісні метрики precision, recall, MRR або nDCG не заявляються без еталонного набору запитів. Основні сценарії наведено в таблиці 3.20.

Таблиця 3.21 – Матриця функціональної перевірки

| Сценарій      | Дія                                       | Очікуваний результат                                          |
|---------------|-------------------------------------------|---------------------------------------------------------------|
| Вхід          | Коректні й помилкові облікові дані        | JWT і рольовий маршрут або контрольована відмова              |
| Заявка        | Signup, approve, reject                   | Pending; створення User тільки після approve                  |
| Синхронізація | Додати, змінити, видалити документ        | Оновлення локального стану відповідно до фактичного алгоритму |
| Парсинг       | DOCX, PDF, TXT, CSV, scan PDF             | Текст або контрольована помилка; OCR не очікується            |
| Пошук         | Змістовий, порожній і нерелевантний запит | Результати, [] або валідаційна відповідь без падіння UI       |
| Граф          | Папки, результати, довгі назви, resize    | Стабільні вузли, зв'язки та modal                             |
| Ролі          | Прямі запити до заборонених endpoint-ів   | Backend повертає заборону незалежно від frontend              |

### 3.9 Інструкція користувача

Користувач відкриває EduGraph, вводить логін і пароль. Якщо облікового запису немає, він подає заявку та очікує рішення уповноваженого користувача.

Після входу користувач відкриває сторінку пошуку, вводить змістовий запит і очікує появи document-вузлів. Клік по вузлу відкриває фрагмент; кнопка в modal переходить до оригіналу в Google Drive.

Teacher, Admin або SuperAdmin може опрацювати заявки, керувати дозволеними типами користувачів і поставити синхронізацію в чергу. Відповідь 202 означає прийняття запиту, а не підтвердження завершення sync.

Для ефективного використання EduGraph потрібно підтримувати впорядковану структуру Google Drive. Назви папок повинні бути змістовними, оскільки вони використовуються у графі. Якщо папки мають випадкові або технічні назви, графова навігація буде менш корисною для користувача.

Документи мають бути у форматах, які система може прочитати. DOCX, PDF, TXT і CSV підтримуються, однак скановані PDF без текстового шару можуть не дати придатного вмісту. Якщо корпус містить багато сканів, потрібно додати OCR-етап або підготувати текстові версії документів.

Для остаточного оформлення інструкції потрібно додати такі відсутні скриншоти:

*[МІСЦЕ ДЛЯ СКРИНШОТА: стан виконання пошуку з індикатором завантаження]*

*[МІСЦЕ ДЛЯ СКРИНШОТА: панель Teacher/Admin із кнопкою запуску Google Drive sync]*

*[МІСЦЕ ДЛЯ СКРИНШОТА: стан порожнього результату пошуку]*

*[МІСЦЕ ДЛЯ СКРИНШОТА: контрольоване повідомлення про помилку пошуку або синхронізації]*

## ВИСНОВКИ

У кваліфікаційній роботі розроблено й описано EduGraph – веб-систему семантичного пошуку в корпусі освітніх документів з інтерактивною графовою навігацією.

В аналітичній частині зіставлено файловий, повнотекстовий, векторний і генеративний підходи, сформовано вимоги та обґрунтовано поділ системи на React-клієнт, ASP.NET Core API і FastAPI-сервіс.

Функціональна частина визначає ролі, варіанти використання й основні сценарії. Повні специфікації винесено до додатка, тому основний текст не перевантажено повторенням кроків.

У проєктній частині описано SQLite-схему без зовнішніх ключів між прикладними сутностями та з технічними зв'язками Identity, FastAPI retrieval-процес, LanceDB, CrossEncoder reranking, клієнтську runtime-модель графу, ASP.NET Core API та алгоритм синхронізації Google Drive.

Фактична реалізація підтримує витягування тексту з DOCX, PDF, TXT і CSV, індексацію chunks, пошук за embeddings, уточнення порядку результатів, відображення документів у графі та перехід до першоджерела.

Окремо зафіксовано обмеження: відсутність OCR і PPTX-парсера, неатомарний upsert LanceDB, відсутність benchmark-оцінювання та неповна перевірка результатів vector-search під час синхронізації. Ці обмеження не приховуються й визначають напрями подальшого розвитку.

## ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. EduGraph: вихідний код проєкту. URL: <https://github.com/ilyaghrischenko/EduGraph> (дата звернення: 06.06.2026).
2. Manning C. D., Raghavan P., Schütze H. Introduction to Information Retrieval. URL: <https://nlp.stanford.edu/IR-book/> (дата звернення: 06.06.2026).
3. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. URL: <https://aclanthology.org/D19-1410/> (дата звернення: 06.06.2026).
4. Wang L. et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training. URL: <https://arxiv.org/abs/2212.03533> (дата звернення: 06.06.2026).
5. Microsoft Learn. Tutorial: Create a Minimal API with ASP.NET Core. URL: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/min-web-api?view=aspnetcore-9.0> (дата звернення: 06.06.2026).
6. Microsoft Learn. SQLite EF Core Database Provider. URL: <https://learn.microsoft.com/en-us/ef/core/providers/sqlite/> (дата звернення: 06.06.2026).
7. Microsoft Learn. Introduction to Identity on ASP.NET Core. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity> (дата звернення: 06.06.2026).
8. React. React Reference Overview. URL: <https://react.dev/reference/react> (дата звернення: 06.06.2026).
9. Vite. Getting Started. URL: <https://vite.dev/guide/> (дата звернення: 06.06.2026).
10. react-force-graph. React component for force-directed graphs. URL: <https://github.com/vasturiano/react-force-graph> (дата звернення: 06.06.2026).
11. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/> (дата звернення: 06.06.2026).
12. Google for Developers. Google Drive API overview. URL: <https://developers.google.com/workspace/drive/api/guides/about-sdk> (дата звернення: 06.06.2026).

13. LanceDB Documentation. Vector Search. URL: <https://docs.lancedb.com/search/vector-search> (дата звернення: 06.06.2026).
14. Sentence Transformers Documentation. Semantic Search. URL: [https://www.sbert.net/examples/sentence\\_transformer/applications/semantic-search/README.html](https://www.sbert.net/examples/sentence_transformer/applications/semantic-search/README.html) (дата звернення: 06.06.2026).
15. Nogueira R., Cho K. Passage Re-ranking with BERT. URL: <https://arxiv.org/abs/1901.04085> (дата звернення: 06.06.2026).
16. LangChain. RecursiveCharacterTextSplitter. URL: [https://docs.langchain.com/oss/python/integrations/splitters/recursive\\_text\\_splitter](https://docs.langchain.com/oss/python/integrations/splitters/recursive_text_splitter) (дата звернення: 06.06.2026).
17. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT). RFC 7519. URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата звернення: 06.06.2026).
18. OpenAPI Initiative. OpenAPI Specification. URL: <https://spec.openapis.org/oas/latest.html> (дата звернення: 06.06.2026).
19. Microsoft Learn. Get the contents of a document part from a package. URL: <https://learn.microsoft.com/en-us/office/open-xml/general/how-to-get-the-contents-of-a-document-part-from-a-package> (дата звернення: 06.06.2026).
20. UglyToad. PdfPig: read and extract text and other content from PDFs in C#. URL: <https://github.com/UglyToad/PdfPig> (дата звернення: 06.06.2026).

## ДОДАТКИ

### Додаток А. Специфікації основних сценаріїв

У додатку наведено специфікації сценаріїв, перелічених у підрозділі 2.2.

Таблиця А.1 – Специфікація UC-01 – Увійти до системи

| Поле                    | Зміст                                                                                                                      |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Основний актор          | Користувач із наявним обліковим записом                                                                                    |
| Передумови              | Відкрита сторінка входу.                                                                                                   |
| Основний потік          | Користувач вводить логін і пароль; API перевіряє Identity; повертається JWT з role claim; frontend відкриває маршрут ролі. |
| Альтернативи та помилки | Порожні або неправильні дані; відсутній користувач; недійсний токен.                                                       |
| Постумова               | Користувач отримує тільки дозволений доступ.                                                                               |

Таблиця А.2 – Специфікація UC-02 – Подати заявку на реєстрацію

| Поле                    | Зміст                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------------|
| Основний актор          | Неавторизований користувач                                                                     |
| Передумови              | Відкрита сторінка реєстрації.                                                                  |
| Основний потік          | Користувач заповнює форму; API перевіряє дані й створює SignUpApplication зі статусом Pending. |
| Альтернативи та помилки | Зайнятий логін; активна заявка з таким логіном; помилка валідації.                             |
| Постумова               | Заявка доступна уповноваженим ролям.                                                           |

Таблиця А.3 – Специфікація UC-03 – Опрацювати заявку

| Поле                    | Зміст                                                                                                                |
|-------------------------|----------------------------------------------------------------------------------------------------------------------|
| Основний актор          | Teacher, Admin або SuperAdmin                                                                                        |
| Передумови              | Існує Pending-заявка.                                                                                                |
| Основний потік          | Користувач обирає approve або reject. Approve створює Identity User і роль; reject змінює статус без створення User. |
| Альтернативи та помилки | Заявка відсутня або вже оброблена; помилка Identity.                                                                 |
| Постумова               | Статус стає Approved або Rejected.                                                                                   |



Таблиця А.4 – Специфікація UC-04 – Виконати семантичний пошук

| Поле                    | Зміст                                                                                                                                 |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Основний актор          | Авторизований користувач                                                                                                              |
| Передумови              | API і FastAPI доступні; vector store містить chunks.                                                                                  |
| Основний потік          | Frontend надсилає Query; backend викликає /search; FastAPI створює query embedding, виконує cosine retrieval, фільтрацію й reranking. |
| Альтернативи та помилки | Порожній запит; відсутні результати; помилка FastAPI.                                                                                 |
| Постумова               | Frontend отримує документи й будує document-вузли.                                                                                    |

Таблиця А.5 – Специфікація UC-05 – Переглянути фрагмент і оригінал

| Поле                    | Зміст                                                                                            |
|-------------------------|--------------------------------------------------------------------------------------------------|
| Основний актор          | Авторизований користувач                                                                         |
| Передумови              | Граф містить document-вузол.                                                                     |
| Основний потік          | Користувач натискає вузол; відкривається DocumentModal; безпечне посилання веде до Google Drive. |
| Альтернативи та помилки | URL не проходить перевірку; користувач закриває modal.                                           |
| Постумова               | Дані не змінюються; користувач отримує контекст результату.                                      |

Таблиця А.6 – Специфікація UC-06 – Запустити синхронізацію

| Поле                    | Зміст                                                                                                                           |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Основний актор          | Teacher, Admin, SuperAdmin або фонові служба                                                                                    |
| Передумови              | Налаштовано Google Drive і service account.                                                                                     |
| Основний потік          | Запит потрапляє в чергу; сервіс оновлює папки й документи, видаляє відсутні записи та надсилає NotIndexed документи до FastAPI. |
| Альтернативи та помилки | Черга заповнена; помилка Drive, parsing або vector-search.                                                                      |
| Постумова               | Локальний стан оновлено відповідно до фактично виконаних операцій.                                                              |

## Додаток Б. Текстова специфікація API та моделей даних

Додаток Б подає текстову специфікацію API та моделей даних. Його призначення полягає в тому, щоб винести частину технічних деталей з основного тексту й залишити їх доступними для перевірки. Основна частина записки пояснює архітектуру та проєктні рішення, а додаток деталізує, які саме групи endpoint-ів і моделей використовуються в реалізації.

Специфікація не замінює OpenAPI-документацію, але є зручною для пояснювальної записки, оскільки описує endpoint-и людською мовою. Вона також допомагає показати зв'язок між frontend, backend і vector-search. Якщо в майбутньому API зміниться, цей додаток потрібно буде синхронізувати з фактичним openapi.json і кодом endpoint-ів.

У таблиці наведено не всі технічні деталі HTTP-відповідей, а основні групи дій, які є важливими для функціональної моделі. Для дипломної роботи цього достатньо, оскільки мета полягає не у повному копіюванні Swagger-опису, а в поясненні призначення API в архітектурі EduGraph.

Таблиця Б.1 – Групи маршрутів API EduGraph

| Група       | Маршрут / дія                     | Призначення                                                         |
|-------------|-----------------------------------|---------------------------------------------------------------------|
| Auth        | POST auth/login                   | Перевірка логіна й пароля, формування JWT з роллю користувача.      |
| Auth        | POST auth/signup                  | Створення заявки на реєстрацію без негайного створення користувача. |
| GoogleDrive | GET google-drive/folders          | Повернення списку папок для побудови графу.                         |
| GoogleDrive | GET google-drive/root-folder-link | Повернення посилання на кореневу папку Google Drive.                |
| GoogleDrive | GET google-drive/search-documents | Семантичний пошук документів через vector-search сервіс.            |
| GoogleDrive | POST                              | Запуск синхронізації                                                |

|                              |                          |                                            |
|------------------------------|--------------------------|--------------------------------------------|
|                              | google-drive/sync        | корпусу документів.                        |
| SignUpApplications           | GET pending applications | Перегляд заявок, які очікують рішення.     |
| SignUpApplications           | POST approve / reject    | Підтвердження або відхилення заявки.       |
| Students / Teachers / Admins | GET / POST / DELETE      | Керування користувачами відповідного типу. |

Endpoint auth/login реалізує сценарій входу. Він приймає логін і пароль, виконує валідацію, шукає користувача, перевіряє пароль через Identity і повертає JWT. У цьому endpoint важливо, що backend не повертає зайвих даних користувача. Роль передається всередині token, а frontend читає її через auth utilities.

Endpoint auth/signup реалізує не створення користувача, а створення заявки. Це принципова відмінність від простих систем реєстрації. Заявка стає проміжною сутністю, яку має обробити уповноважена роль. Такий endpoint не повинен автоматично надавати доступ, інакше була б порушена модель контрольованого доступу до освітніх документів.

Endpoint google-drive/folders потрібний для графового інтерфейсу. Він повертає папки, які frontend перетворює на folder-вузли. Цей endpoint важливий навіть до пошуку, тому що базовий граф структури корпусу може бути побудований без результатів semantic search. Папки також використовуються для прив'язки знайдених документів за folderName.

Endpoint google-drive/root-folder-link повертає посилання на головну папку. Frontend кешує це посилання, щоб не виконувати зайвий запит кожного разу. Посилання потрібне для кнопки відкриття кореневого сховища. Це дає користувачу швидкий доступ до джерельної структури Google Drive.

Endpoint google-drive/search-documents є центральним для користувацького сценарію. Він приймає Query як параметр, перевіряє його, формує VectorSearchRequest і викликає VectorSearchService. Після отримання

результатів backend мапить VectorSearchResult у власний Response. Це дозволяє відокремити внутрішній контракт FastAPI від frontend-контракту.

Endpoint google-drive/sync запускає синхронізацію. Він не повинен виконувати всю довгу роботу прямо в request-response сценарії без контролю, тому в системі передбачені фонові служби й черга примусового запуску. Для користувача це виглядає як дія «оновити корпус», але всередині запускається складний процес.

Endpoint-и SignUpApplications відповідають за адміністративний життєвий цикл заявок. GetPendingApplications повертає заявки, які очікують рішення. Approve створює користувача та змінює статус заявки. Reject змінює статус без створення користувача. Усі ці дії повинні бути захищені політиками ролей.

Endpoint-и Students, Teachers і Admins реалізують керування користувачами. Вони можуть мати схожу структуру, але різні політики доступу. Такий поділ робить API зрозумілим: кожна група endpoint-ів відповідає за певний тип користувача. Для frontend це також зручно, оскільки можна мати окремі api-модулі studentsApi, teachersApi і adminsApi.

Взаємодія frontend з API відбувається через apiFetch. Це означає, що endpoint-и не викликаються довільним fetch у кожній сторінці. Централізація запитів важлива для авторизації, обробки 401, налаштування base URL і єдиного підходу до JSON. У пояснювальній записці це можна подати як приклад зменшення дублювання на клієнтському рівні.

Взаємодія backend з FastAPI відбувається через VectorSearchService. Цей сервіс приховує HTTP-деталі й повертає Result. Якщо FastAPI недоступний або повертає помилку, backend отримує ErrorDetails. Це не усуває помилку, але робить її керованою та придатною для відображення або запису в статус індексації.

Таблиця Б.2 – Основні моделі даних API

| Модель | Ключові поля | Коментар |
|--------|--------------|----------|
|--------|--------------|----------|

|                          |                                                                |                                                                      |
|--------------------------|----------------------------------------------------------------|----------------------------------------------------------------------|
| SearchDocuments.Response | Title, Url, FolderName, Content, Score, ChunkId, ChunkIndex    | Контракт між backend і frontend для відображення результатів пошуку. |
| VectorSearchRequest      | query, top_k, min_score, min_rerank_score                      | Контракт між ASP.NET Core API та FastAPI-сервісом.                   |
| VectorDocumentDto        | id, title, content, url, content_hash, folder_name             | Дані, які backend передає для індексації документа.                  |
| IncomingDocument         | id, title, content, url, content_hash, folder_name             | Pydantic-модель FastAPI для upsert.                                  |
| SearchResult             | document_id, chunk_id, chunk_index, title, content, url, score | Результат FastAPI після vector search і reranking.                   |

SearchDocuments.Response є моделлю, яку frontend фактично використовує для побудови результатів. Title відображається як назва документа. Url використовується для переходу до Google Drive. FolderName дозволяє прив'язати результат до папки. Content показується в modal. Score, ChunkId і ChunkIndex потрібні для технічної ідентифікації та пояснення результату.

VectorSearchRequest є моделлю запиту від backend до FastAPI. У простому випадку backend передає тільки Query, але FastAPI також підтримує top\_k, min\_score і min\_rerank\_score. Це означає, що в майбутньому backend може надати користувачу або адміністратору можливість налаштовувати кількість результатів і пороги релевантності.

VectorDocumentDto є моделлю документа для індексації. Вона передає id, title, content, url, contentHash і folderName. Ця модель не містить усіх полів UniversityDocument, а тільки ті, які потрібні vector-search сервісу. Це правильне рішення, оскільки Python-сервіс не повинен знати про всі доменні деталі backend.

IncomingDocument у FastAPI є Pydantic-відображенням документа. Для title і content встановлено обмеження max\_length. Це захищає сервіс від надто великих або некоректних запитів. Хоча такі обмеження не замінюють повноцінну політику ресурсів, вони є першим рівнем контролю розміру вхідних даних.

SearchRequest містить query, top\_k, min\_score і min\_rerank\_score. Значення top\_k обмежене, щоб користувач не міг запросити надто велику видачу. min\_score дозволяє відсіювати слабкі vector matches. min\_rerank\_score дозволяє відкинути результати, які CrossEncoder оцінив надто низько.

SearchResult містить як vector score, так і rerank score. Це важливо для діагностики якості пошуку. Vector score показує близькість embeddings, а rerank score показує оцінку пари query-content. У frontend зараз основний акцент робиться на результаті й фрагменті, але ці оцінки можуть бути використані для майбутнього аналізу.

UniversityDocument є доменною моделлю, яка зв'язує Google Drive і vector index. GoogleDriveId використовується як стабільний зовнішній ідентифікатор. ContentHash показує, чи змінився текст. SearchIndexStatus показує, чи відповідає vector index поточному тексту. SearchIndexError дозволяє не втрачати інформацію про проблеми індексації.

UniversityFolder потрібна для графу та збереження структури Google Drive. Вона має IsMain, що дозволяє відокремити кореневу папку від звичайних тематичних папок. Це важливо, бо root-вузол графу не є звичайною папкою з точки зору UI: він виконує роль центральної точки навігації.

User і SignUpApplication відображають організаційну частину системи. EduGraph не є анонімним пошуковиком, тому модель користувачів має бути пов'язана з ролями. Заявка дозволяє створювати користувачів контрольовано, а не через самостійну відкриту реєстрацію.

GraphNode і GraphLink на frontend не є backend-моделями, але вони важливі для інтерфейсу. GraphNode може представляти root, trunk, folder або

document. GraphLink з'єднує вузли. Така внутрішня модель дозволяє перетворити список папок і список документів у графову структуру без зміни backend API.

### **Додаток В. Матриця ролей, вимог і сценаріїв перевірки**

Додаток В деталізує рольову модель і сценарії перевірки. У головному тексті ролі описані концептуально, а тут вони представлені як матриця доступу. Такий формат зручний для перевірки відповідності між вимогами, frontend-маршрутами та backend-політиками.

Матриця не повинна сприйматися як остаточний юридичний регламент доступу. Вона описує технічну модель дипломного прототипу. Якщо система буде впроваджуватися в реальному університетському середовищі, ролі й права потрібно буде погодити з відповідальними особами.

Таблиця В.1 – Матриця доступу за ролями

| Роль        | Пошук | Заявки | Користувачі | Синхронізація | Особливості                                        |
|-------------|-------|--------|-------------|---------------|----------------------------------------------------|
| Student     | Так   | Ні     | Ні          | Ні            | Основний сценарій – пошук і перегляд результатів.  |
| Teacher     | Так   | Так    | Частково    | За політикою  | Може брати участь у керуванні навчальним доступом. |
| Admin       | Так   | Так    | Так         | Так           | Основна адміністративна роль.                      |
| Super Admin | Так   | Так    | Так         | Так           | Додатково керує адміністраторами.                  |

Роль Student має найвужчий набір прав. Вона потрібна для користувача, який шукає матеріали та переглядає результати. Студент не повинен керувати заявками, користувачами або синхронізацією, оскільки ці дії впливають на стан системи.

Роль Teacher має проміжний характер. У межах прототипу викладач може мати доступ до частини адміністративних сценаріїв, зокрема заявок або створення користувачів. Це відповідає ситуації, коли викладач бере участь у супроводі навчального доступу. Остаточний набір прав цієї ролі може бути уточнений перед впровадженням.

Роль Admin призначена для повного супроводу користувачів і корпусу документів. Адміністратор може працювати зі списками, заявками й синхронізацією. Ця роль повинна бути обмежена, оскільки неправильні дії адміністратора можуть вплинути на доступ користувачів або актуальність індексу.

Роль SuperAdmin відрізняється можливістю керувати адміністраторами. Це потрібно для відокремлення звичайного адміністрування від найвищого рівня керування системою. У невеликому прототипі така роль може виглядати надлишковою, але для реального середовища вона корисна.

Матриця доступу повинна перевірятися на двох рівнях. На backend рівні перевіряються policies. На frontend рівні перевіряються routes і навігація. Якщо frontend приховує сторінку, але backend дозволяє запит, це потенційна проблема. Якщо backend забороняє запит, але frontend показує сторінку, це UX-проблема. Обидва рівні мають бути узгоджені.

Рольова модель також впливає на тестування. Для кожної ролі потрібно перевірити дозволені й заборонені маршрути. Наприклад, Student має відкривати /student/search, але не /admin/admins. SuperAdmin має відкривати адміністративні сторінки. Teacher не повинен випадково отримати доступ до дій, які мають бути тільки для SuperAdmin.



Таблиця В.2 – Сценарії функціональної перевірки

| ID   | Сценарій                 | Передумова                            | Очікуваний результат                          |
|------|--------------------------|---------------------------------------|-----------------------------------------------|
| T-01 | Успішний<br>вхід         | Користувач існує й пароль правильний. | Повертається JWT, відкривається маршрут ролі. |
| T-02 | Невдалий<br>вхід         | Пароль неправильний.                  | Доступ не надається, показується помилка.     |
| T-03 | Подання<br>заявки        | Користувач заповнив форму.            | Створюється pending-заявка.                   |
| T-04 | Підтвердженн<br>я заявки | Заявка очікує рішення.                | Створюється користувач з роллю.               |
| T-05 | Синхронізаці<br>я        | Google Drive доступний.               | Папки й документи оновлюються в SQLite.       |
| T-06 | Індексація               | Є NotIndexed документи.               | Створюються chunk-и у LanceDB.                |
| T-07 | Пошук                    | Індекс містить документи.             | Повертаються ranked результати.               |
| T-08 | Граф                     | Frontend отримав folders і docs.      | Будуються вузли й links, modal відкривається. |

Сценарій T-01 перевіряє базову працездатність login. Він важливий, бо всі інші сценарії залежать від авторизації. Якщо token не формується або роль не читається, користувач не зможе правильно перейти до свого маршруту. Успішний результат означає, що backend, Identity, JWT і frontend auth utilities працюють узгоджено.

Сценарій T-02 перевіряє відмову при неправильних даних. Система не повинна уточнювати, що саме неправильне – login чи password, оскільки це

може допомагати перебору. Достатньо повідомити, що дані неправильні. Також не повинно створюватися жодного токена.

Сценарії T-03 і T-04 перевіряють розділення заявки та користувача. Це одна з важливих бізнес-вимог. Якщо заявка одразу створює користувача, модель контрольованого доступу порушена. Якщо підтвердження заявки не створює користувача, сценарій реєстрації не завершується.

Сценарій T-05 перевіряє роботу із зовнішнім джерелом. Google Drive може містити папки, документи різних форматів і вкладені структури. Система повинна обійти дерево, отримати текст і посилання, а не тільки назви файлів. Також потрібно перевіряти видалення.

Сценарій T-06 перевіряє зв'язок SQLite і LanceDB. Якщо документ має статус `NotIndexed`, він повинен бути переданий у FastAPI. Після успішного `upsert` статус повинен змінитися на `Indexed`. Якщо статус залишається `NotIndexed` після успішного `upsert`, система буде повторювати індексацію.

Сценарій T-07 перевіряє `semantic search`. Успішним результатом є не просто HTTP 200, а наявність змістовних полів у відповіді. Для frontend потрібні `title`, `url`, `content`, `folderName`, `chunkId` і `chunkIndex`. Якщо будь-яке з ключових полів відсутнє, UI може працювати неправильно.

Сценарій T-08 перевіряє граф. Граф повинен містити `root`, папки й документи. Якщо документ має `folderName`, він повинен бути прив'язаний до відповідної папки. Якщо `folderName` відсутній, документ не повинен втрачатися, а має бути прив'язаний до `root`. Modal повинен відкривати фрагмент саме того документа, по якому натиснув користувач.

Ці сценарії можна виконувати вручну під час демонстрації або поступово автоматизувати. Для backend доцільні `integration tests`, для frontend – `browser/e2e` перевірки, для `vector-search` – HTTP tests з невеликим контрольним набором документів. У межах поточної записки вони фіксують план перевірки й показують, що тестування не обмежується візуальним оглядом інтерфейсу.

## Додаток Г. Розгорнута інструкція та чек-лист супроводу

Додаток Г містить розгорнуту інструкцію та чек-лист супроводу системи. Основна інструкція користувача в розділі 3.9 описує роботу коротко, щоб не перевантажувати головний текст. У цьому додатку ті самі процеси деталізовано з погляду користувача, адміністратора та особи, яка супроводжує систему після розгортання.

Додаток корисний з двох причин. По-перше, він показує, що система EduGraph розглядається не тільки як програмний код, а як інструмент, яким мають користуватися реальні ролі. По-друге, він фіксує експлуатаційні обмеження, які можуть бути неочевидними під час демонстрації: якість документів, доступність Google Drive, стан індексації та ресурси Python-сервісу.

Цей додаток доповнює основну інструкцію користувача й показує практичний порядок використання системи після розгортання. Він потрібний для того, щоб відокремити короткий опис у розділі 3.9 від розгорнутих експлуатаційних кроків і чек-листів супроводу.

Таблиця Г.1 – Послідовність роботи користувача

| Крок | Дія користувача                | Реакція системи                                |
|------|--------------------------------|------------------------------------------------|
| 1    | Відкрити сторінку входу.       | Показується форма логіна й пароля.             |
| 2    | Увести облікові дані.          | Backend перевіряє користувача й повертає JWT.  |
| 3    | Перейти до пошуку.             | Frontend відкриває маршрут відповідно до ролі. |
| 4    | Увести змістовий запит.        | API передає запит до vector-search сервісу.    |
| 5    | Натиснути на документ у графі. | Відкривається modal з релевантним фрагментом.  |
| 6    | Відкрити оригінал.             | Браузер переходить до Google Drive документа.  |

Користувач студентського типу починає роботу зі сторінки входу. Якщо він не має облікового запису, він не повинен звертатися безпосередньо до адміністратора поза системою, а може подати заявку. Такий підхід централізує процес отримання доступу та залишає слід у вигляді заявки. Для навчального середовища це зручніше, ніж ручне створення облікових записів без історії.

Після входу студент бачить пошуковий інтерфейс. Його не потрібно навчати технічним поняттям *embeddings*, *vector score* або *reranking*. Інтерфейс має бути сформульований навколо простої дії: ввести запит і переглянути результати. Якщо система працює правильно, студент бачить не тільки документ, а й фрагмент, який пояснює релевантність.

Під час формулювання запиту користувачу варто рекомендувати писати змістовні фрази, а не тільки одне слово. Семантичний пошук краще використовує контекст, якщо запит містить предмет, дію або характеристику. Наприклад, запит «вимоги до оформлення звіту з практики» є інформативнішим, ніж просто «звіт».

Якщо результатів немає, це не завжди означає, що документ відсутній. Можливо, запит сформульовано занадто вузько, документ ще не проіндексовано або текст не був витягнутий з файла. Користувачу можна рекомендувати змінити формулювання, використати ширші поняття або звернутися до адміністратора, якщо очікуваний документ точно має бути в корпусі.

Коли користувач відкриває *modal* документа, він має прочитати фрагмент і вирішити, чи потрібно відкривати повний документ. Система не повинна заохочувати використання фрагмента як повної заміни джерела. Фрагмент є навігаційною підказкою, а не остаточним навчальним матеріалом.

Перехід до Google Drive потрібний для перевірки повного контексту. У документі можуть бути таблиці, рисунки, примітки або попередні розділи, які не потрапили в *chunk*. Тому у навчальному середовищі коректним сценарієм є «знайти через EduGraph – перевірити в оригіналі – використати матеріал».

Викладач або адміністратор працює з системою ширше. Окрім пошуку, він може переглядати заявки. Під час підтвердження заявки потрібно перевіряти, чи справді користувач належить до відповідної групи або має право на роль. Технічна кнопка approve не замінює організаційне рішення.

Під час створення користувачів адміністратор має бути уважним до ролі. Неправильно видана роль може відкрити зайві сторінки або, навпаки, заблокувати потрібний сценарій. Особливо обережно потрібно працювати з роллю SuperAdmin, оскільки вона має найширші права.

Запуск синхронізації слід виконувати після помітних змін у Google Drive. Якщо додано один документ, можна дочекатися фонових процесів. Якщо потрібно швидко перевірити новий матеріал, ручний запуск прискорює оновлення. Водночас не варто запускати синхронізацію багато разів поспіль без потреби, оскільки це створює навантаження на Google Drive API та vector-search сервіс.

Користувачі повинні розуміти, що результат пошуку залежить від індексації. Якщо документ щойно додано, він може з'явитися в Google Drive, але ще не бути в LanceDB. Тому між додаванням документа та його появою в semantic search може бути затримка. Це нормальна властивість системи з окремим індексом.

Таблиця Г.2 – Зони супроводу системи

| Зона супроводу | Що контролювати                                                  | Ознака проблеми                                                  |
|----------------|------------------------------------------------------------------|------------------------------------------------------------------|
| Google Drive   | Структуру папок, доступ service account, формати файлів.         | Папки не оновлюються або документи не читаються.                 |
| SQLite         | Наявність користувачів, заявок, документів, статусів індексації. | Документи не переходять у Indexed або мають Failed.              |
| FastAPI        | Старт моделей, доступність /health, розмір vector_db.            | Пошук повертає 500 або порожній результат для очевидних запитів. |

|          |                                         |                                                                   |
|----------|-----------------------------------------|-------------------------------------------------------------------|
| Frontend | Маршрути, токен, граф, modal, safe URL. | Користувач бачить неправильну сторінку або не відкриває документ. |
|----------|-----------------------------------------|-------------------------------------------------------------------|

Супровід Google Drive починається зі структури папок. Папки повинні мати зрозумілі назви, тому що вони відображаються у графі. Якщо папка названа технічно або випадково, користувач побачить цю назву в інтерфейсі. Тому порядок у Google Drive прямо впливає на якість графової навігації.

Доступ service account має бути стабільним. Якщо обліковий запис, через який backend читає Google Drive, втратить права, синхронізація перестане працювати. У такому випадку frontend може продовжувати показувати старі результати, але корпус не буде оновлюватися. Це потрібно контролювати через логи синхронізації та статуси документів.

Супровід SQLite включає контроль цілісності таблиць і резервне копіювання. Навіть якщо Google Drive є джерелом документів, SQLite містить користувачів, заявки, статуси індексації та локальний стан корпусу. Втрата цієї БД означатиме втрату облікових записів і необхідність повторної синхронізації.

Супровід vector\_db включає контроль розміру, відповідності документам і можливість повторної побудови. У поточному прототипі vector\_db можна розглядати як похідне сховище, яке за потреби можна відтворити з SQLite і Google Drive. Проте для великого корпусу повторна індексація може тривати довго, тому резервне копіювання також може бути доцільним.

FastAPI-сервіс має власні ресурсні вимоги. Моделі SentenceTransformer і CrossEncoder завантажуються в пам'ять, а embeddings і reranking потребують обчислень. Якщо сервер має мало пам'яті або CPU, пошук може бути повільним. Перед production-розгортанням потрібно визначити мінімальні ресурси для стабільної роботи.

Логи є важливою частиною супроводу. Backend повинен фіксувати помилки Google Drive, vector-search і індексації. Python-сервіс фіксує vector result і rerank result, що корисно для діагностики. У майбутньому можна додати централізований збір логів через OpenTelemetry або інший observability stack.

Чек-лист перед фінальним поданням роботи має включати перевірку структури записки, актуальності схем, відповідності назв розділів методичним вимогам, оновлення переліку посилань, перевірку української мови, перегляд рисунків і таблиць, а також підтвердження того, що описані технології відповідають фактичному коду.

Окремо потрібно перевірити, що тема з LLM пояснена коректно. Оскільки поточна реалізація не генерує відповіді чат-моделлю, у тексті має бути чітко зазначено, що використовується LLM-орієнтований retrieval-підхід на основі transformer-моделей embeddings і CrossEncoder reranking. Це усуває можливе протиріччя між назвою теми та реалізацією.

Перед демонстрацією потрібно підготувати контрольний сценарій: запустити backend, FastAPI і frontend; переконатися, що /health працює; виконати login; завантажити папки; виконати пошук; відкрити modal; перейти до Google Drive. Цей сценарій повинен бути коротким і стабільним, щоб під час захисту не залежати від випадкового запиту.

Після захисту система може розвиватися як внутрішній сервіс. Для цього потрібно поступово додати автоматизовані тести, production deployment, резервне копіювання, фільтри, метрики якості пошуку та, за потреби, генеративний LLM-модуль. Поточна архітектура дозволяє робити ці кроки послідовно.

### **Додаток Д. Глосарій термінів і технічних понять**

Глосарій потрібний для уніфікації термінів у пояснювальній записці. Проект EduGraph поєднує веб-розробку, бази даних, Google Drive інтеграцію та методи обробки природної мови. Через це одні й ті самі поняття можуть тлумачитися по-різному. У додатку подано пояснення саме в контексті цієї роботи, а не універсальні академічні визначення.

Особливу увагу потрібно приділити термінам LLM, semantic search, embeddings і reranking. У темі роботи використано формулювання «на основі LLM», але фактична реалізація зосереджена на retrieval-підході. Тому в записці важливо не створювати враження, що система вже генерує відповіді чат-моделлю. Вона використовує transformer-моделі для пошуку та ранжування фрагментів.

Таблиця Д.1 – Основні терміни EduGraph

| Термін            | Пояснення в контексті EduGraph                                                                       |
|-------------------|------------------------------------------------------------------------------------------------------|
| Semantic search   | Пошук, який порівнює зміст запиту й документа через векторні представлення, а не тільки точні слова. |
| Embedding         | Числовий вектор, що представляє зміст текстового фрагмента або запиту.                               |
| Chunk             | Фрагмент документа, який індексується окремо для точнішого пошуку.                                   |
| Reranking         | Другий етап ранжування, де CrossEncoder уточнює порядок кандидатів.                                  |
| Vector store      | Сховище embeddings, у проєкті представлене LanceDB.                                                  |
| JWT               | Токен, що підтверджує автентифікацію користувача й містить роль.                                     |
| VSA               | Vertical Slice Architecture, організація backend за функціональними зрізами.                         |
| Google Drive sync | Процес оновлення локального корпусу на основі папок і документів Google Drive.                       |

Термін «LLM-орієнтований підхід» у цій роботі означає використання сучасних мовних transformer-моделей для представлення та порівняння тексту. Це ширше, ніж класичний keyword search, але вужче, ніж повноцінна



генеративна RAG-система. EduGraph знаходить джерельні фрагменти та показує їх користувачеві, а не створює нову відповідь від імені моделі.

Термін «semantic search» означає пошук за змістом. Якщо користувач вводить запит, система не обмежується перевіркою входження слів у документ. Вона перетворює запит і фрагменти документів у embeddings і порівнює їх у векторному просторі. Завдяки цьому система може знаходити фрагменти, які близькі за змістом, навіть якщо формулювання не збігається повністю.

Embedding є числовим представленням тексту. Для людини embedding нечитабельний, але для алгоритму він дозволяє вимірювати близькість між запитом і фрагментом. У EduGraph embeddings створюються моделлю `intfloat/multilingual-e5-base`. Для документів використовується префікс `passage`, а для запиту – `query`, що відповідає логіці моделі.

Chunk є частиною документа. Поділ на chunk-и потрібний тому, що освітні документи можуть бути великими й містити багато тем. Якщо індексувати весь документ як один вектор, релевантний фрагмент може загубитися серед нерелевантного тексту. Chunking дозволяє знайти конкретний змістовий уривок.

Overlap між chunk-ами потрібний для збереження контексту на межах фрагментів. Якщо текст розрізаний рівно по межі chunk-а, важлива частина речення може опинитися в сусідньому фрагменті. Overlap 150 символів зменшує цей ризик і робить embeddings стабільнішими для пограничних випадків.

Vector store є сховищем embeddings. У EduGraph цю роль виконує LanceDB. Воно зберігає vector разом із метаданими chunk-а: `document_id`, `title`, `content`, `url`, `folder_name` і `content_hash`. Завдяки цьому після пошуку можна повернути не тільки число релевантності, а й змістовний результат для користувача.

Cosine similarity є способом оцінити близькість векторів. Якщо вектори нормалізовані, cosine distance можна перетворити на score. У FastAPI-сервісі

після пошуку в LanceDB score обчислюється як 1.0 мінус distance. Далі результати нижче min\_score відкидаються.

Reranking є другим етапом пошуку. Перший етап швидко знаходить кандидатів через vector similarity, але він може помилятися в порядку. CrossEncoder отримує пару query і content та оцінює її точніше. Це повільніше, але застосовується тільки до обмеженого набору кандидатів, тому залишається практичним.

JWT є токеном, який підтверджує вхід користувача. У EduGraph token містить роль, яку frontend використовує для маршрутизації, а backend – для політик доступу. Якщо token недійсний або прострочений, користувач повинен повернутися на сторінку входу.

Vertical Slice Architecture означає організацію backend не навколо великих шарів controllers/services/models для всього застосунку, а навколо окремих функцій. Кожен endpoint має власні типи й handler. Для EduGraph це зручно, бо функції auth, заявки, користувачі та Google Drive мають різні правила.

Result pattern означає, що метод може повернути успішний результат або ErrorDetails без використання винятку як звичайного способу керування потоком. Це робить бізнес-помилки явними. Наприклад, неправильний пароль або невалідний документ є очікуваними ситуаціями, які можна повернути як Result failure.

Google Drive sync означає не просте копіювання файлів, а процес синхронізації стану. Система читає папки, документи, текст, link і hash, порівнює з локальною БД, створює нові записи, оновлює змінені й видаляє ті, що зникли. Після цього змінені документи індексуються у vector store.

Graph navigation означає подання результатів як вузлів і зв'язків. У EduGraph root-вузол відповідає головному сховищу, folder-вузли – папкам, а document-вузли – знайденим документам. Така модель допомагає бачити не тільки релевантність, а й місце документа в структурі корпусу.

Modal document preview означає перегляд фрагмента без переходу з пошукового інтерфейсу. Це важливо, бо користувач може швидко оцінити, чи справді результат корисний. Якщо фрагмент підходить, користувач відкриває повний документ у Google Drive.

ContentHash є способом визначити зміну документа. Якщо текст змінився, hash також змінюється, і документ потрібно переіндексувати. Це дозволяє не будувати vector index заново для всього корпусу при кожній синхронізації.

SearchIndexStatus показує стан документа щодо vector index. NotIndexed означає, що документ потребує індексації. Indexed означає, що він успішно доданий до індексу. Failed означає, що індексація завершилася помилкою, яку потрібно проаналізувати.

Таким чином, глосарій допомагає однаково використовувати терміни в усій записці. Це особливо важливо для міждисциплінарної теми, де поєднуються програмна інженерія, веб-архітектура та методи семантичного пошуку.